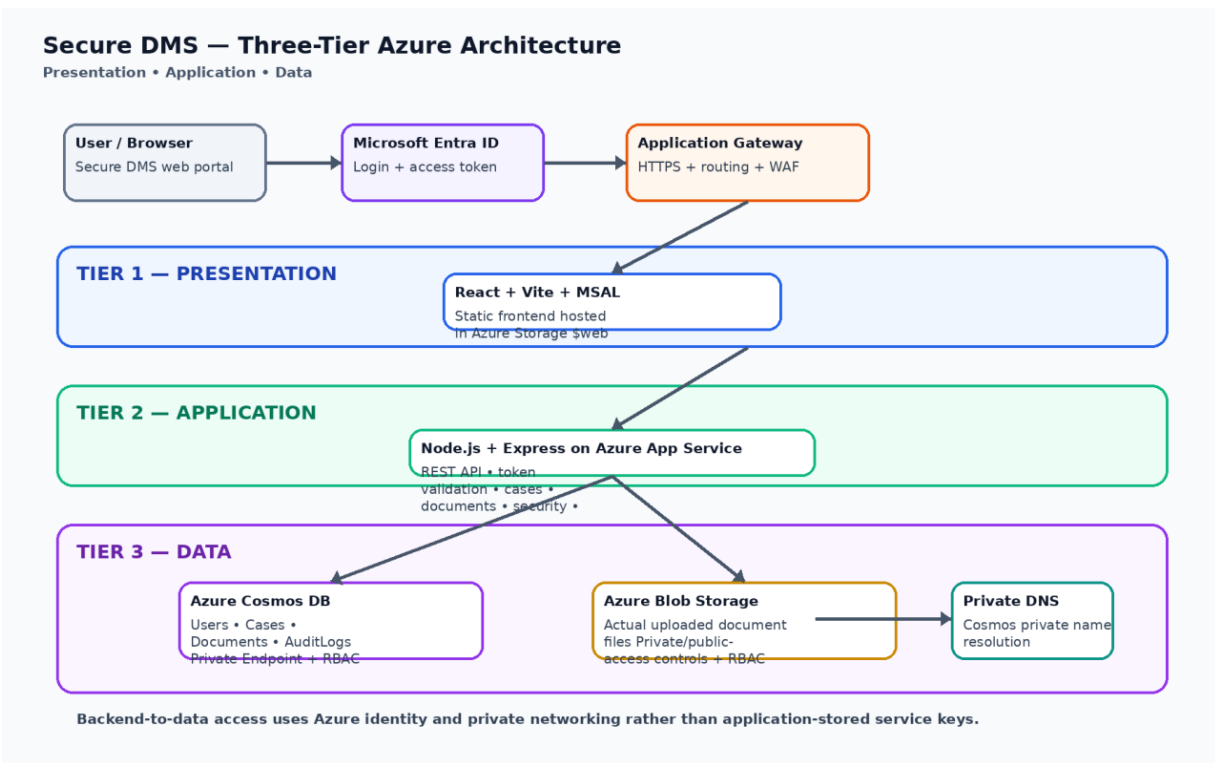


SECURE DIGITAL DOCUMENT MANAGEMENT PORTAL

Azure Capstone Project — Technical Documentation



Prepared by Hariprasath

React • Node.js • Microsoft Entra ID • Azure App Service • Cosmos DB • Blob Storage

1. Architecture at a Glance

Secure DMS follows a three-tier application structure. The presentation tier contains the React web application, the application tier contains the Node.js/Express API hosted on Azure App Service, and the data tier contains Azure Cosmos DB and Azure Blob Storage. Microsoft Entra ID provides identity, while Application Gateway provides the protected public entry point.

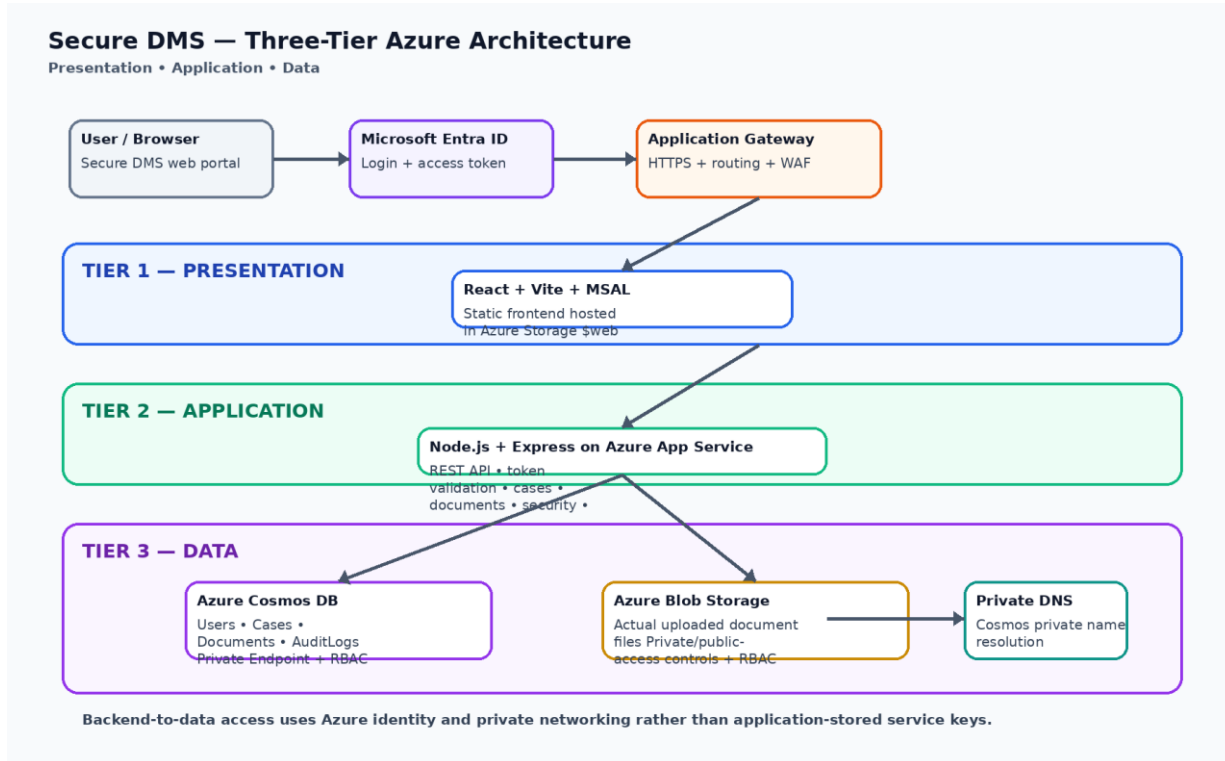


Figure 1. Three-tier architecture of the deployed Secure DMS solution.

The important design idea is separation of responsibility: the browser does not directly manage Azure data resources, the backend controls business operations, and the data services are protected with Azure networking and identity controls.

1.1 Main Request Path

1. User opens the Secure DMS website through the public HTTPS endpoint.
2. Microsoft Entra ID authenticates the user and the frontend obtains an API access token.
3. Application Gateway receives HTTPS traffic and applies routing and WAF protection.
4. The React frontend is served from Azure Storage, while `/api/*` requests are routed to App Service.
5. The backend validates the token and performs the requested case or document operation.
6. Cosmos DB stores structured application data and Blob Storage stores the actual document files.

2. Our Website — Secure Digital Document Management Portal

Secure DMS is a web application for managing digital case records and the documents attached to those cases. The main goal is simple: an authenticated user can enter the portal, work with cases, upload and retrieve documents, and perform sensitive operations through a controlled workflow.

2.1 What We Built

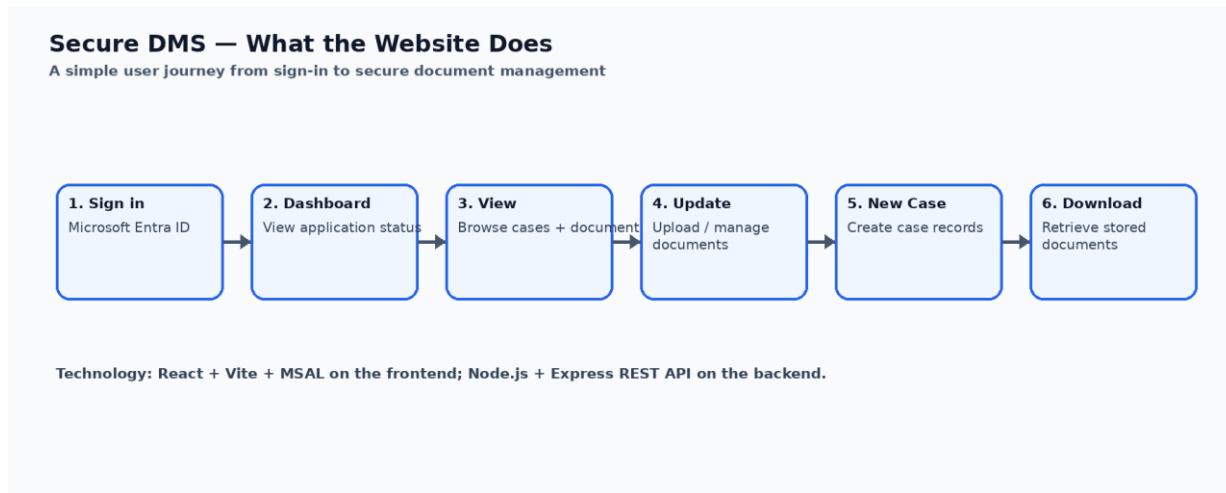


Figure 2. Simplified Secure DMS website user journey.

The website is designed as a single-page application. After sign-in, the user reaches the dashboard and can move between the main modules without leaving the application.

2.2 Technologies Used in the Website

Technology	How it is used
React	Builds the user interface and application pages.
Vite	Builds and packages the React frontend for deployment.
MSAL	Connects the frontend to Microsoft Entra ID for sign-in and access tokens.
JavaScript / JSX / CSS	Implements the application behaviour and visual interface.
Node.js + Express	Provides the backend REST API used by the website.

2.3 What the User Can Do

- Sign in through Microsoft Entra ID.
- Open the dashboard after successful authentication.
- View available cases and inspect the documents associated with a case.
- Create new case records through the NEW CASE module.
- Upload documents through the UPDATE workflow.
- View and download stored documents.
- Use step-up re-authentication before sensitive operations.
- Generate and maintain audit records for important operations.

SECURE DIGITAL DOCUMENT MANAGEMENT PORTAL

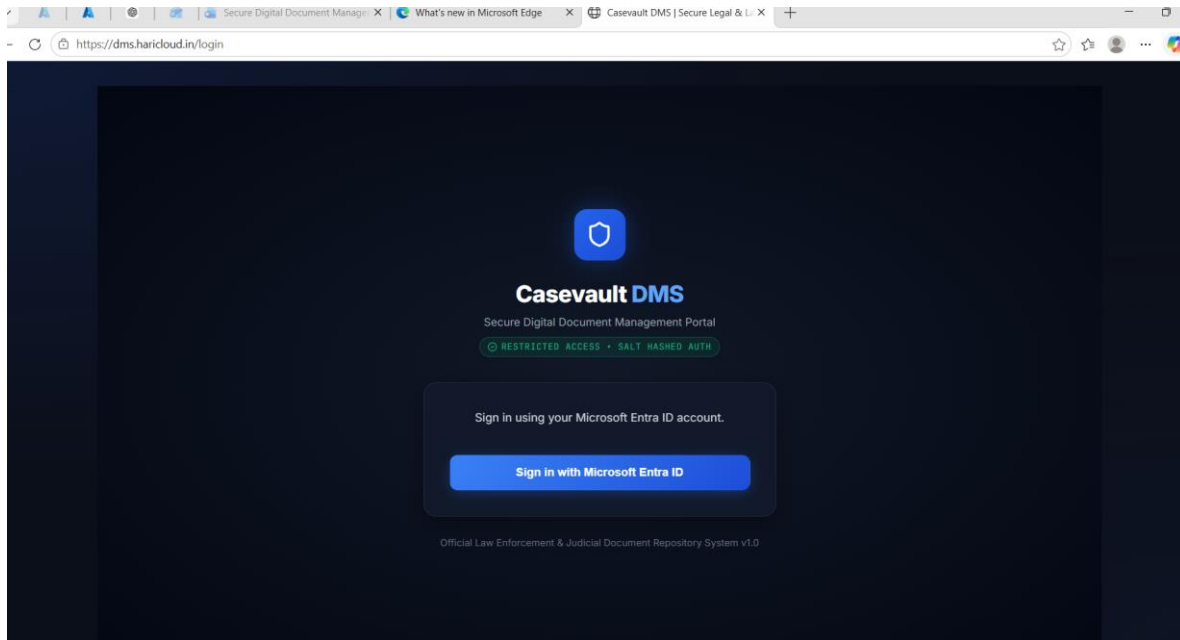


Figure 3. Secure DMS login page.

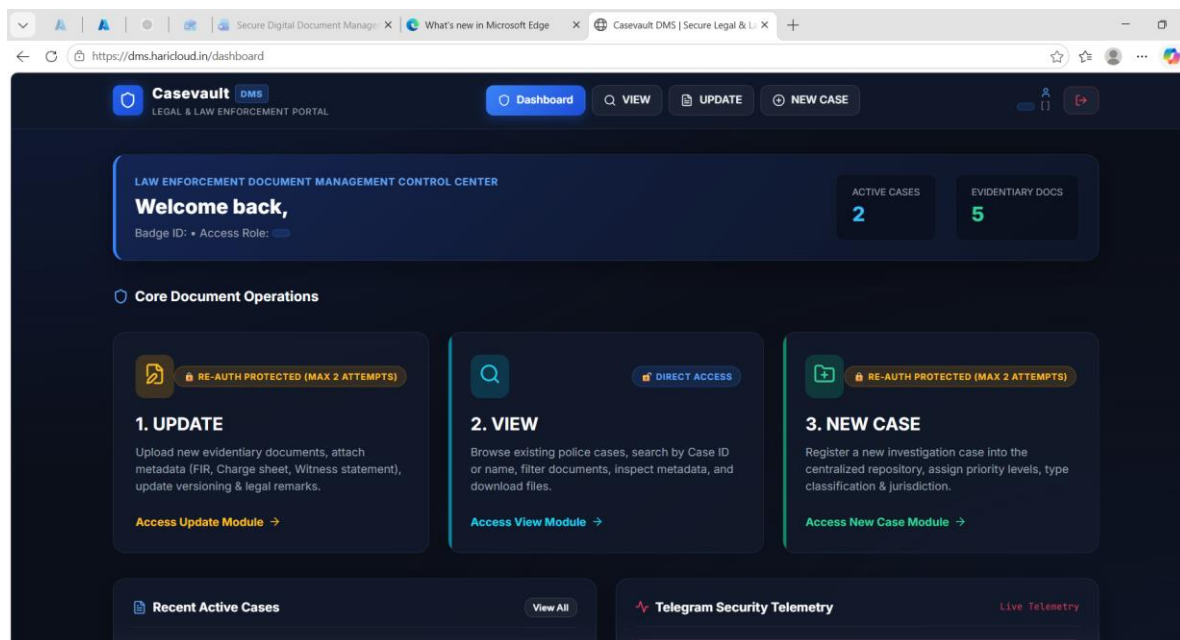


Figure 4. Secure DMS authenticated dashboard.

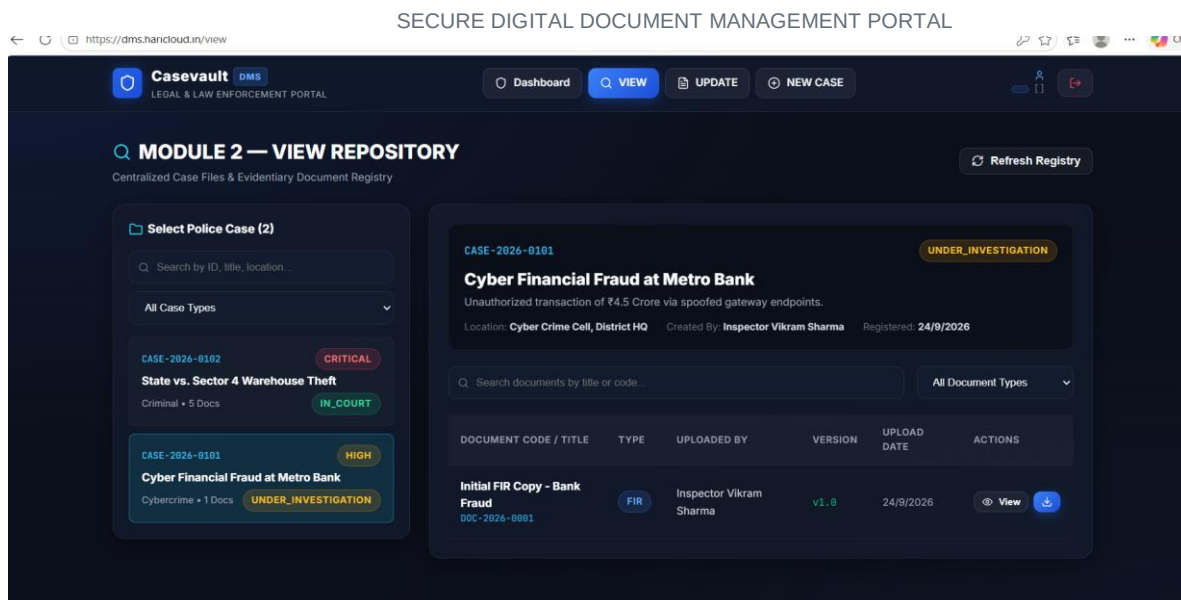


Figure 5. Secure DMS case management screen.

3. How the Website Works Internally

3.1 Frontend

The frontend is a React/Vite single-page application. MSAL handles the Microsoft Entra sign-in flow. After authentication, the frontend obtains an access token for the backend API and sends it with protected API requests.

3.2 Backend

The backend is a Node.js and Express REST API. It is responsible for authentication validation, user lookup, case operations, document operations, security/re-authentication and audit logging. The production backend runs in Azure App Service.

3.3 Data

Structured information such as users, cases, document metadata and audit events is stored in Cosmos DB. The actual uploaded files are stored separately in Azure Blob Storage. This keeps file content and application metadata clearly separated.

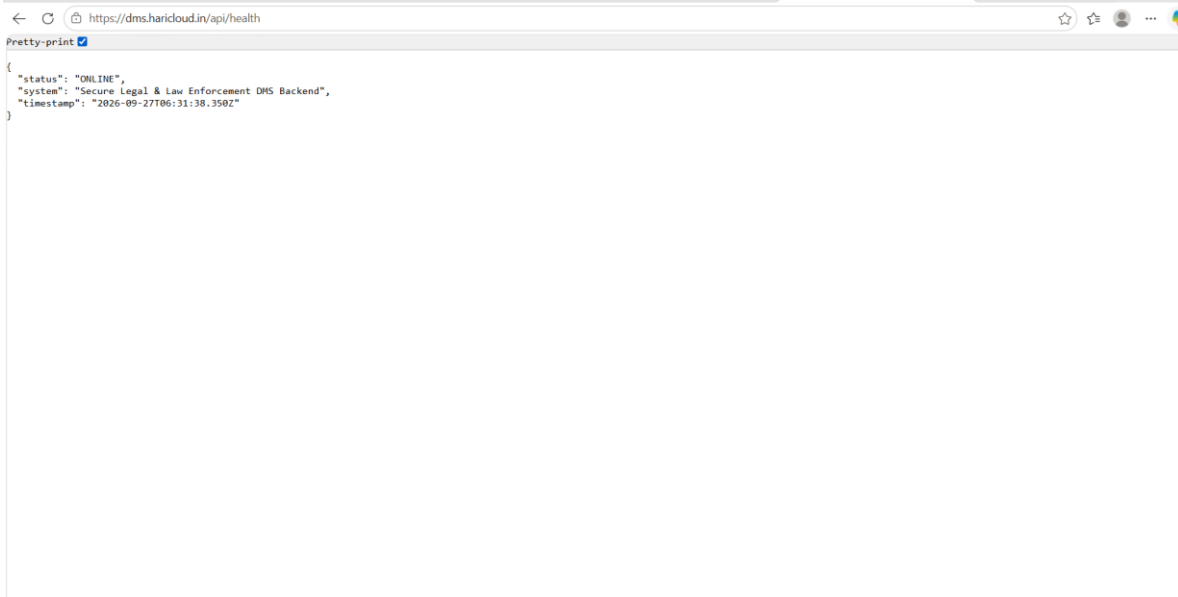


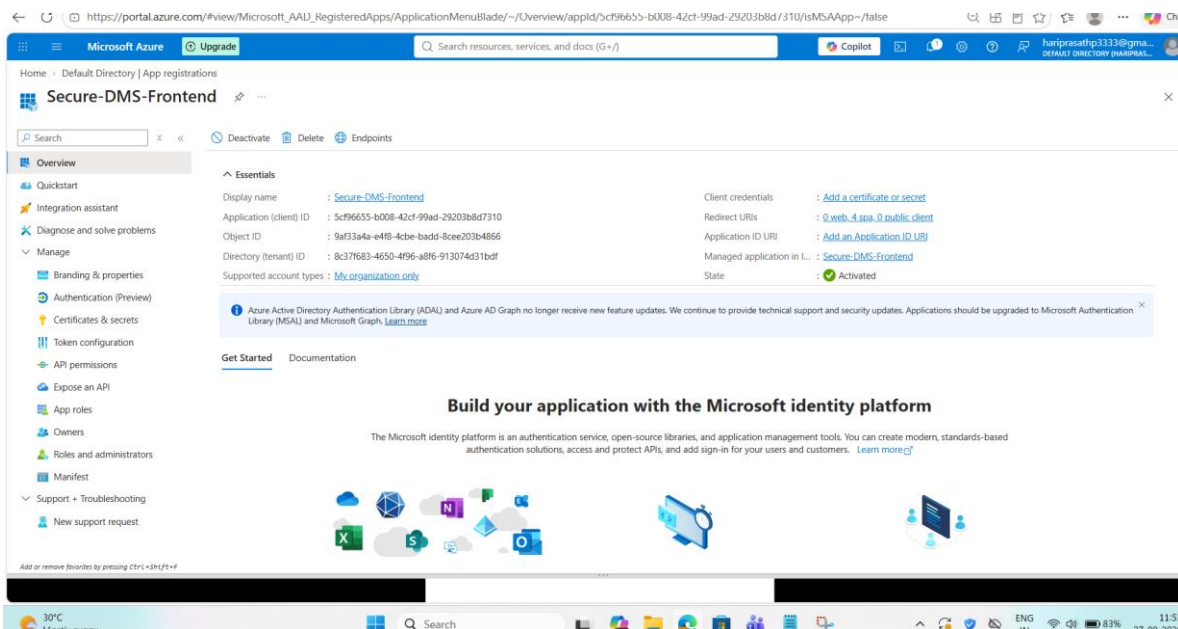
Figure 6. Production backend health endpoint.

4. Azure Services Used in Secure DMS

The Azure environment is not a collection of unrelated services. Each service has one clear job in the application. The following sections explain what we used, why we used it, what we configured, and where the result appears in the deployed system.

4.1 Microsoft Entra ID

Microsoft Entra ID is the identity layer of Secure DMS. We created separate application registrations for the frontend and backend API. The frontend uses MSAL to sign the user in, request the backend API scope, and attach the access token to API requests. The backend validates the token before allowing protected operations.



SECURE DIGITAL DOCUMENT MANAGEMENT PORTAL

Figure 7. Frontend Entra App Registration.

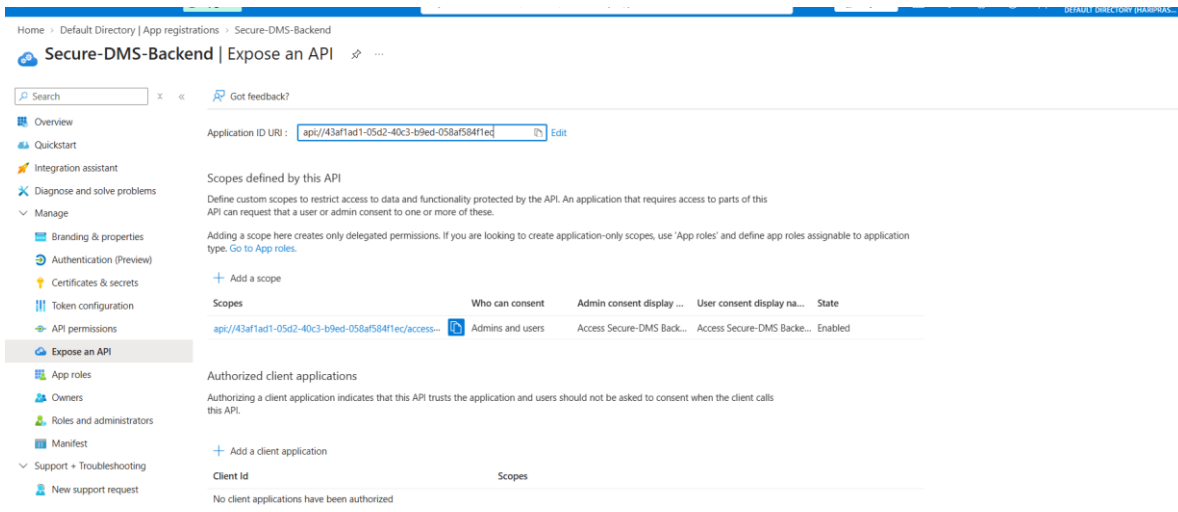


Figure 8. Backend API scope.

4.2 Azure Storage Static Website

Azure Storage hosts the built React frontend. The Vite build output is uploaded to the \$web container and served as the static website. This removes the need to run a separate web server just for the frontend.

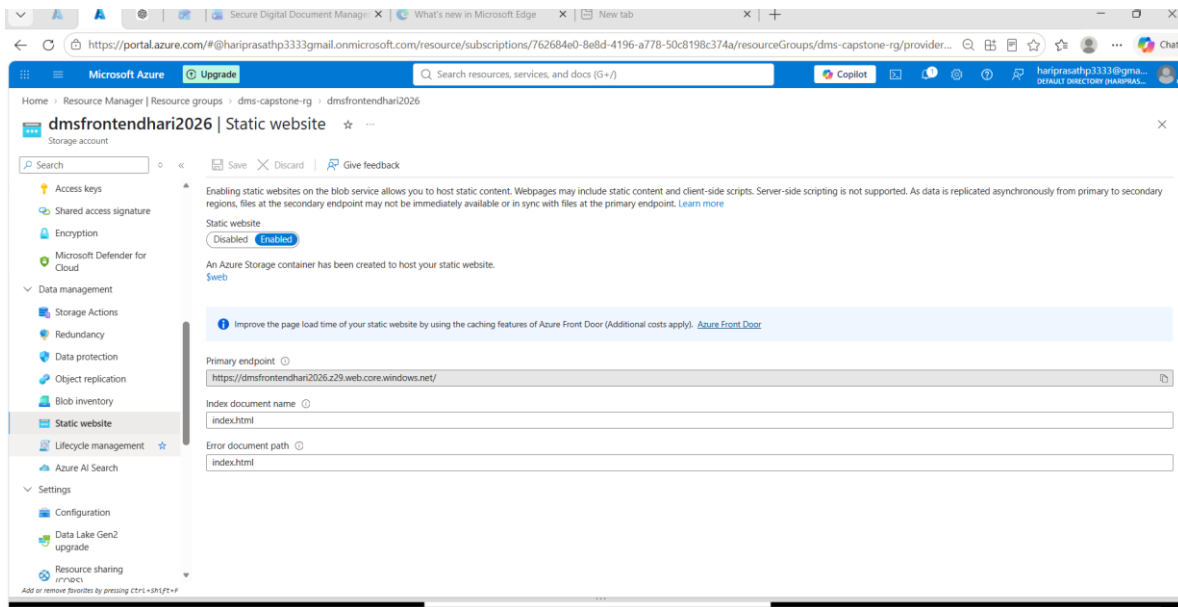


Figure 9. Static website configuration.

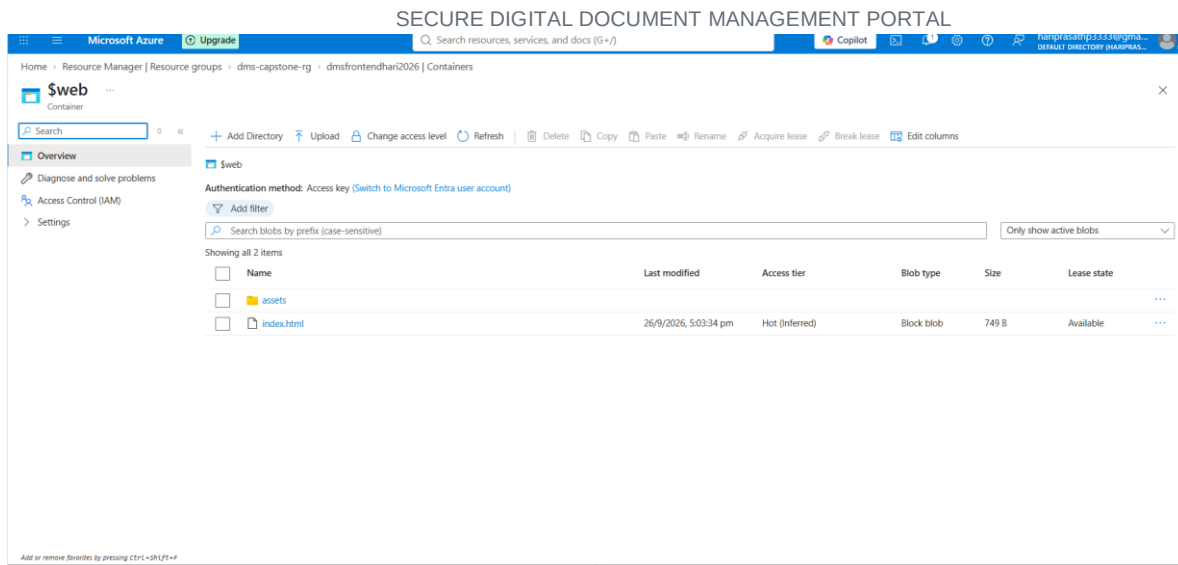


Figure 10. Frontend files in the storage account.

4.3 Azure App Service

The Node.js/Express backend is hosted in dms-backend-appservice-hari. App Service provides the managed compute platform while VNet Integration connects the application to the private Azure network. Access restrictions reduce direct public access to the backend.

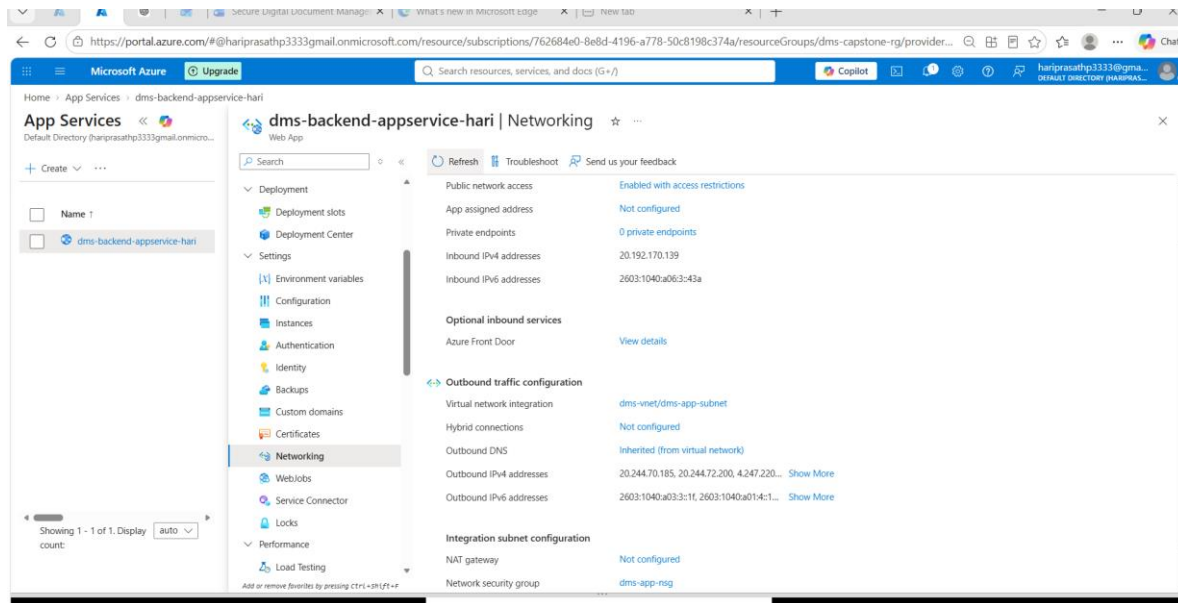


Figure 11. App Service VNet Integration.

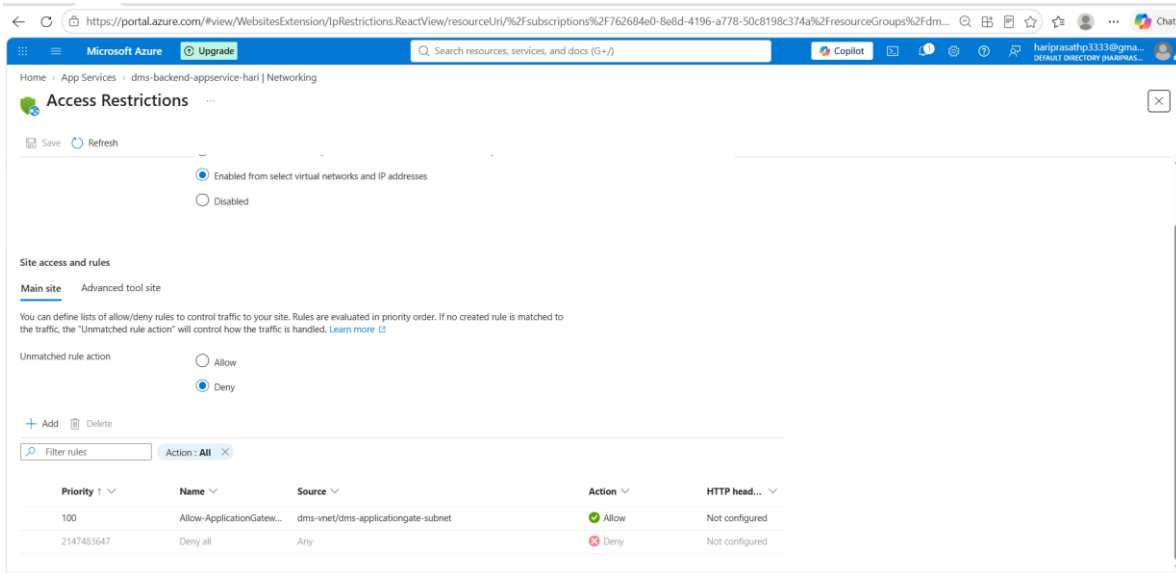


Figure 12. App Service access restrictions.

4.4 Azure Cosmos DB

Cosmos DB is the main application database. We use the dmsdb database with Users, Cases, Documents and AuditLogs containers. Public network access is disabled, a private endpoint is used, and the backend Managed Identity receives the Cosmos DB Built-in Data Contributor role.

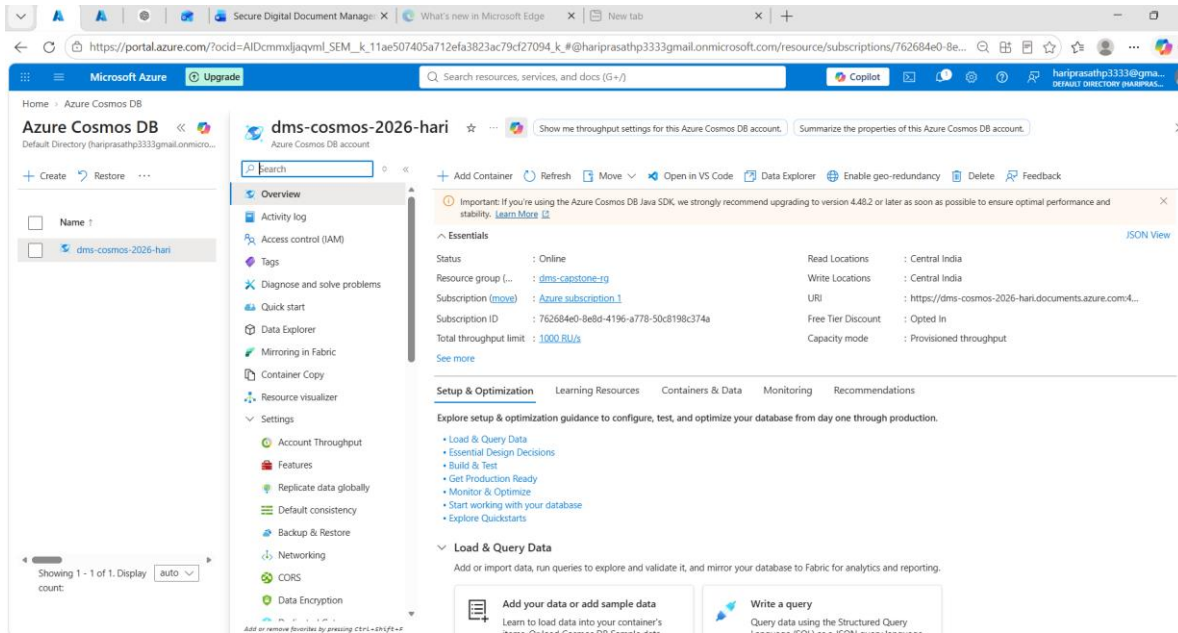


Figure 13. Cosmos DB account.

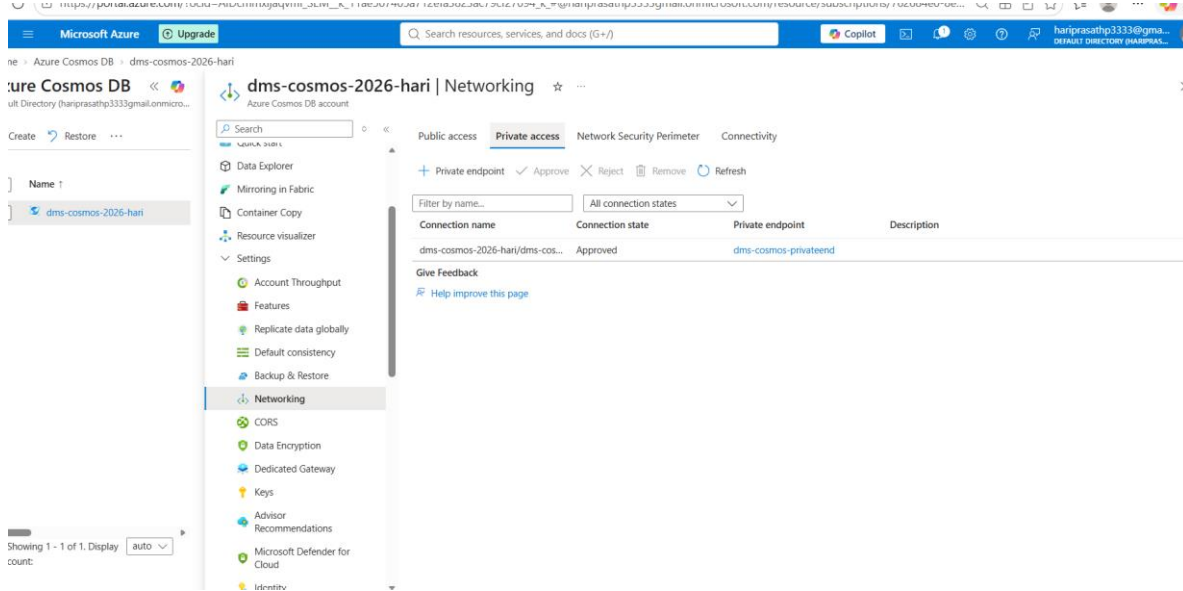


Figure 14. Cosmos DB private endpoint.

4.5 Azure Blob Storage

Blob Storage keeps the actual uploaded document files. The backend uploads and downloads documents through the Azure Storage Blob SDK. Cosmos DB keeps the related metadata such as document ID, case ID, title, filename and version.

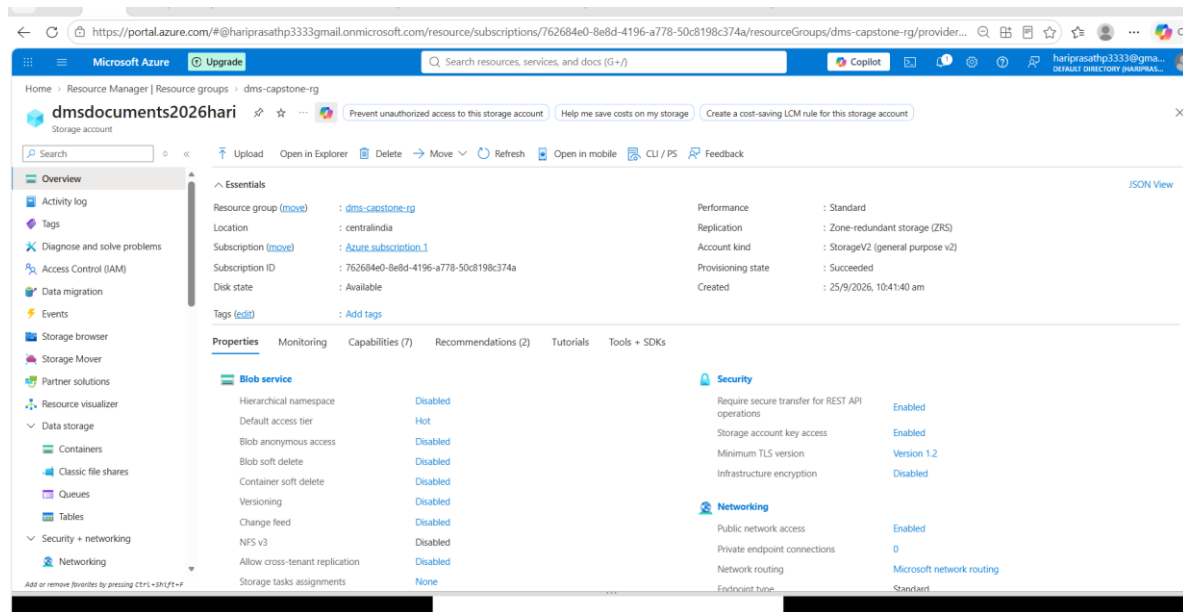


Figure 15. Document storage account.

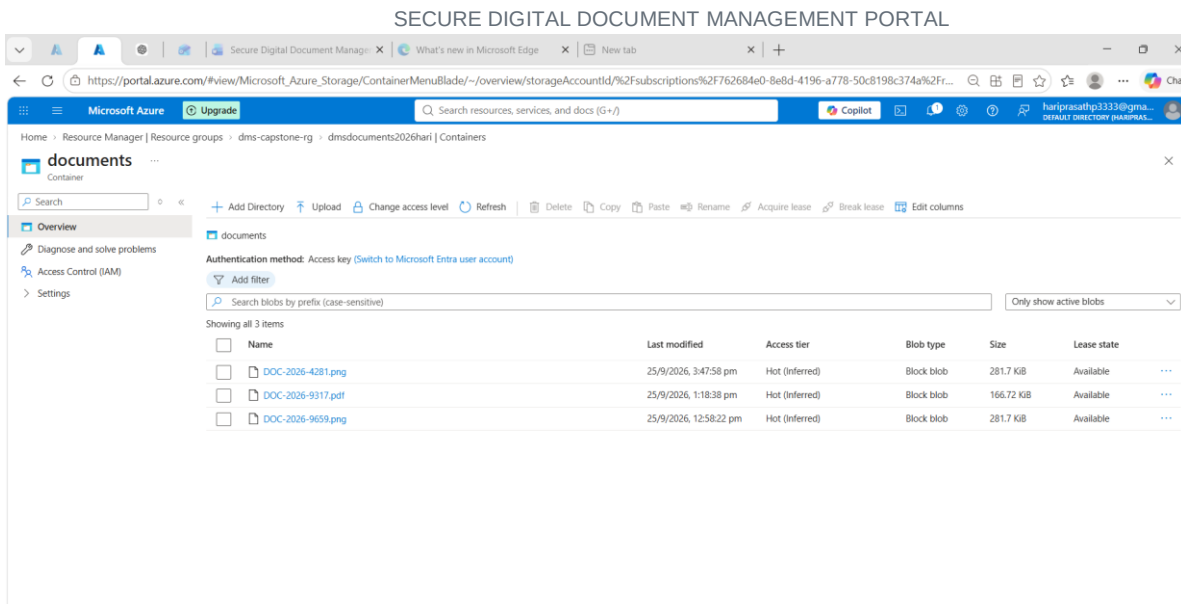


Figure 16. Documents container.

4.6 Azure Virtual Network

dms-vnet provides the private network boundary for the application and data path. The implementation uses separate subnets for App Service integration, Cosmos private endpoint and Application Gateway networking.

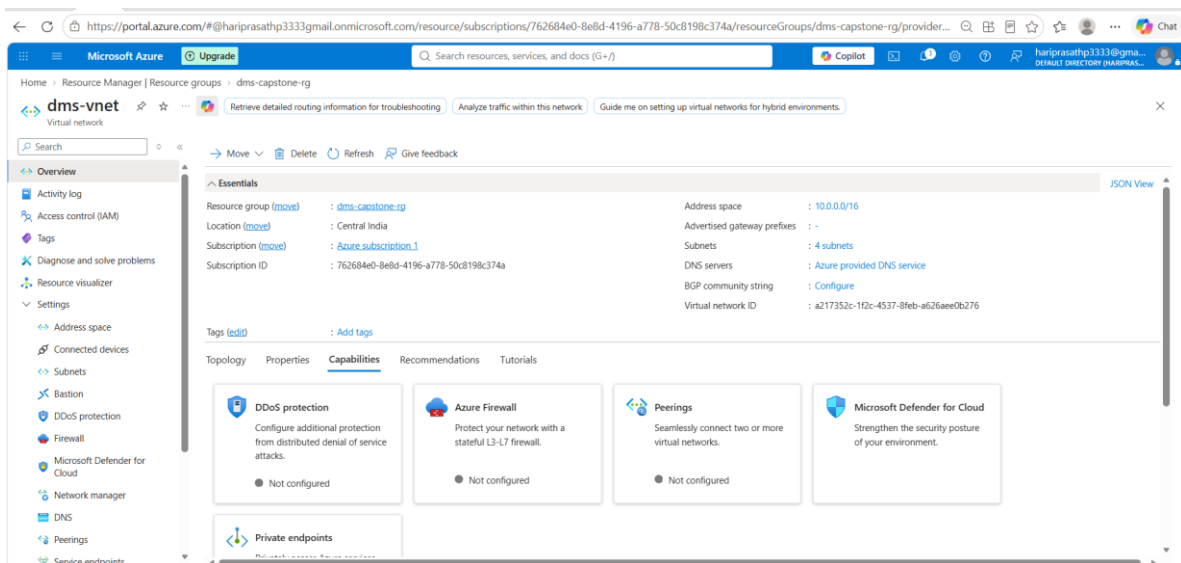


Figure 17. dms-vnet overview.

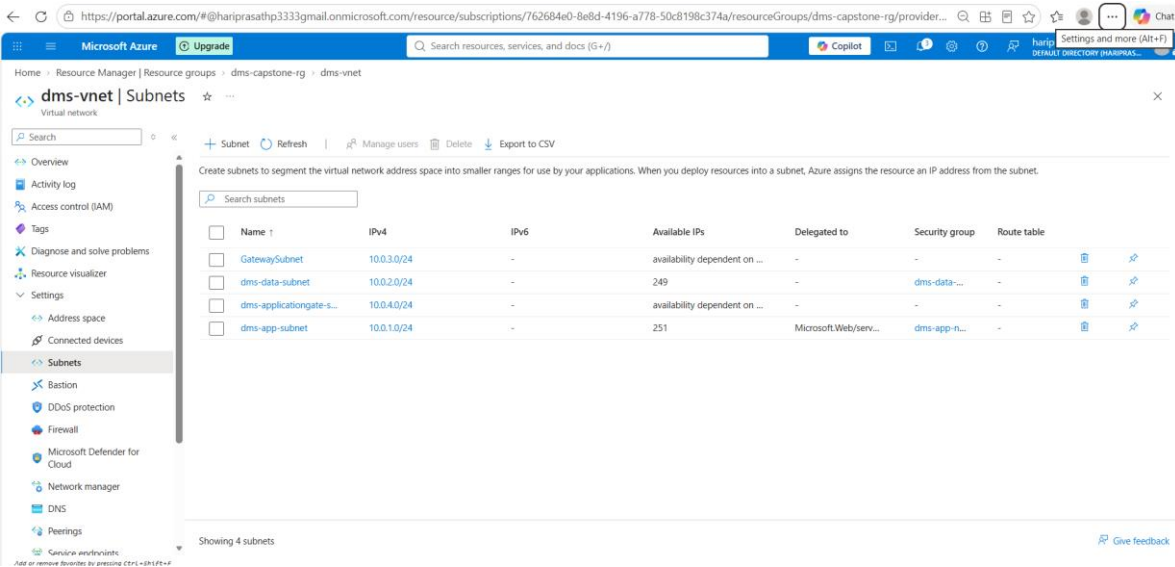


Figure 18. VNet subnets.

4.7 Private Endpoint + Private DNS

The Cosmos DB private endpoint is placed in the data subnet. The private DNS zone `privatelink.documents.azure.com` is linked to the VNet so that the Cosmos endpoint resolves correctly from the private network.

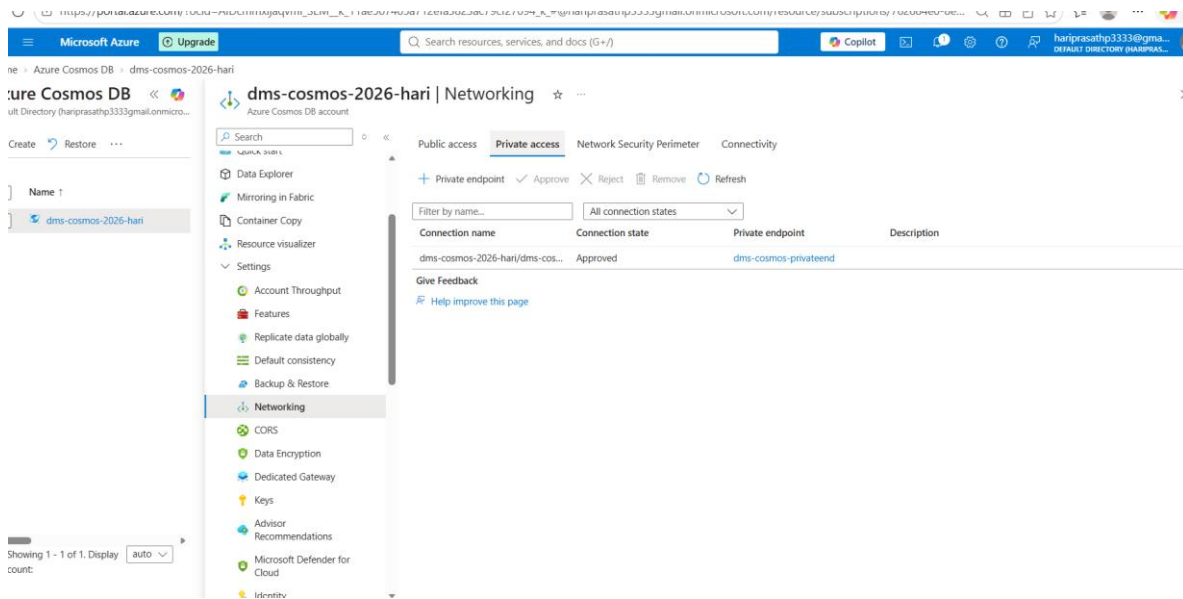


Figure 19. Cosmos DB private endpoint.

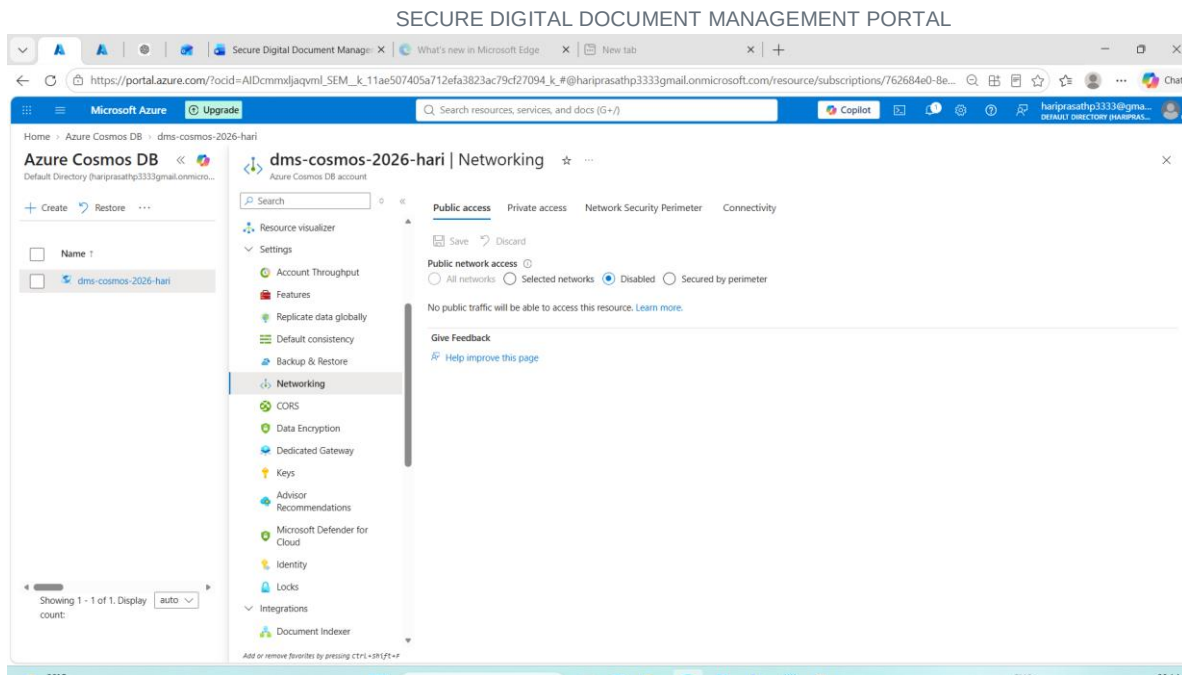


Figure 20. Cosmos DB public network access configuration.

4.8 Application Gateway

Application Gateway is the public Layer-7 entry point. It handles HTTPS, routing and backend health probing. The / path serves the frontend and /api/* is routed to the backend App Service. This also prevents users from treating the backend URL as the normal public entry point.

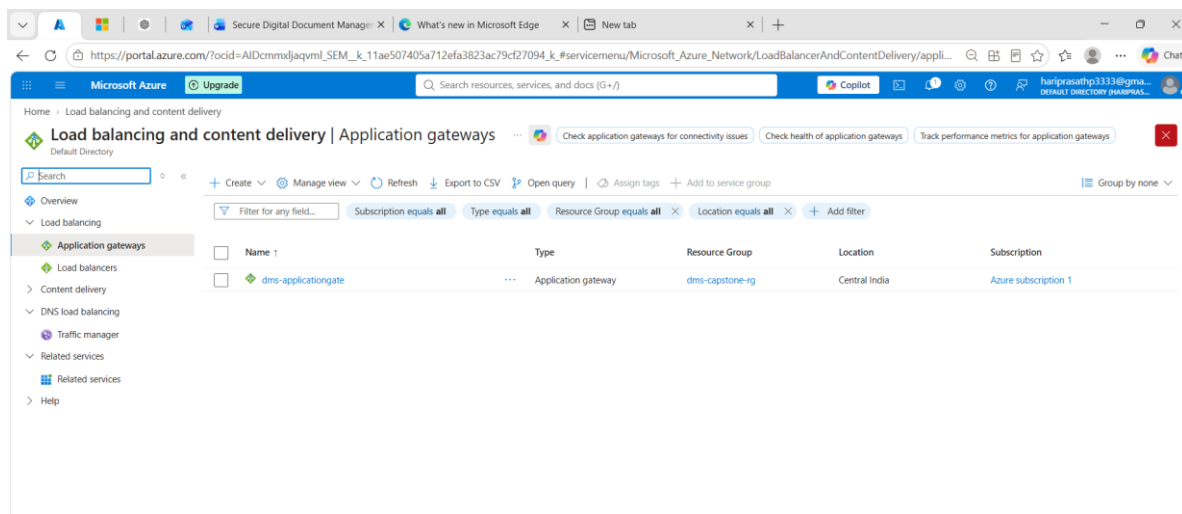


Figure 21. Application Gateway overview.

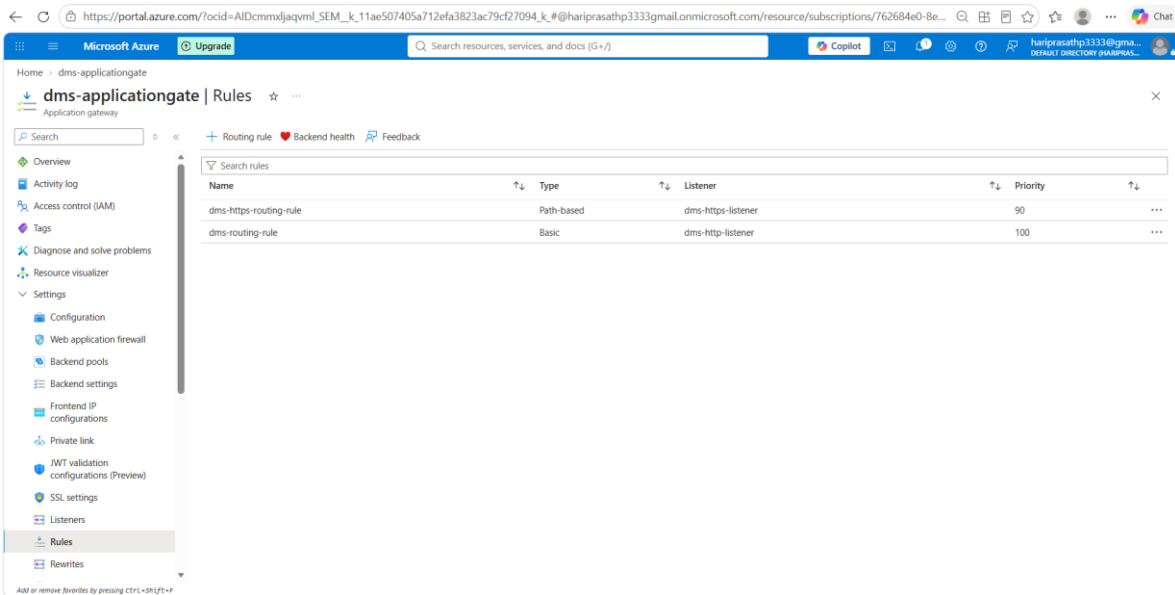


Figure 22. Application Gateway routing rules.

4.9 WAF + HTTPS

The Application Gateway uses WAF_v2. The WAF policy provides an additional security layer at the application entry point. HTTPS is configured with the project certificate and the gateway uses Key Vault integration for certificate access.

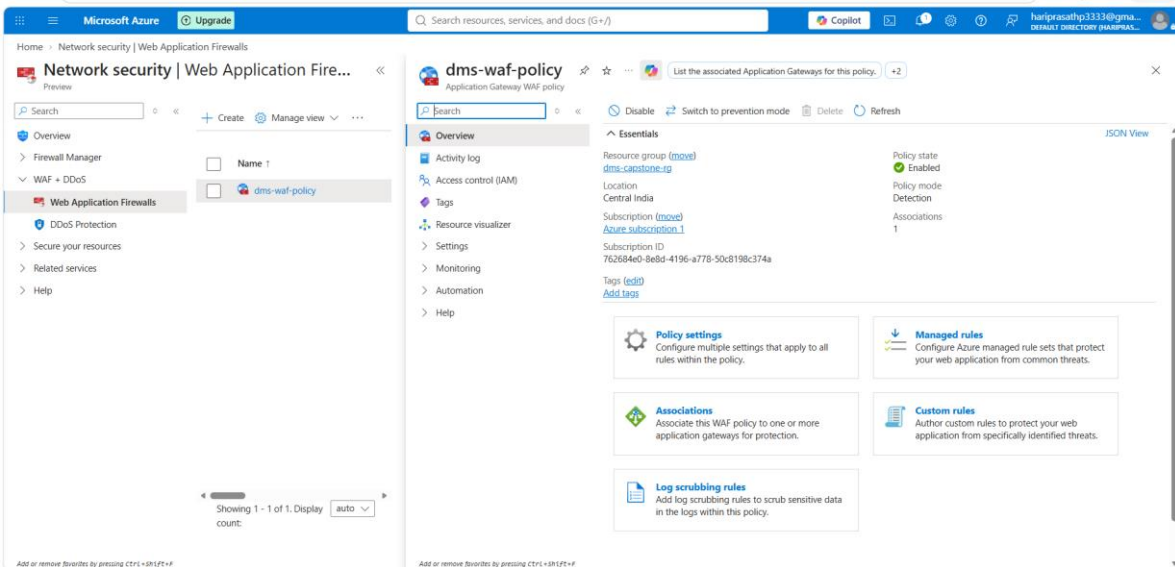


Figure 23. WAF policy.

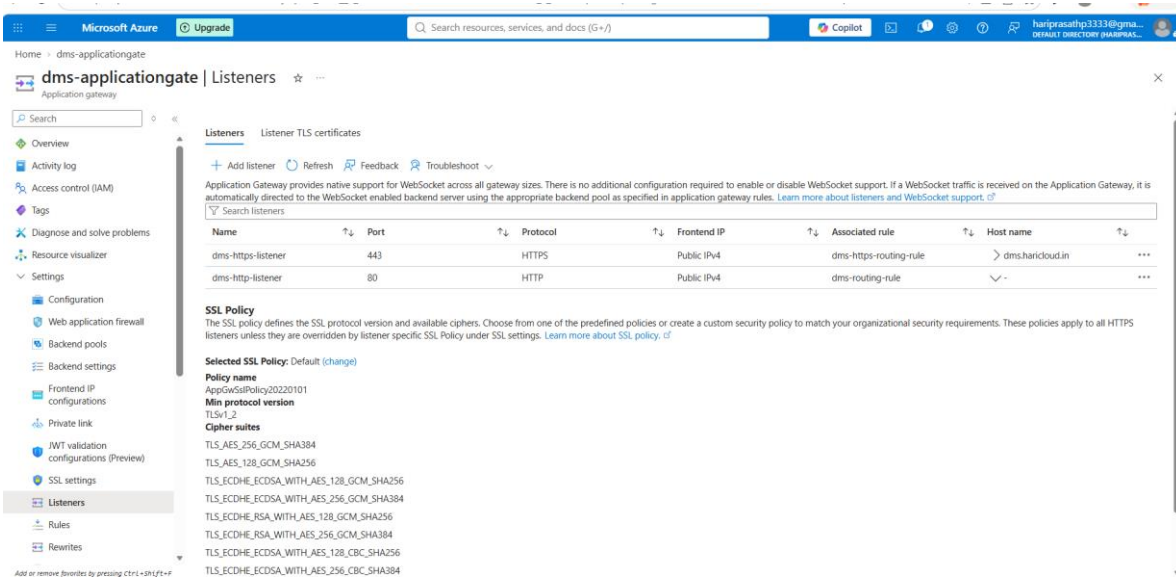


Figure 24. HTTPS listener.

4.10 Managed Identity

The backend uses a system-assigned Managed Identity to authenticate to Azure services. Cosmos DB access is controlled with Cosmos RBAC and Blob Storage access is controlled with Azure RBAC. This avoids putting Azure service credentials in application code.

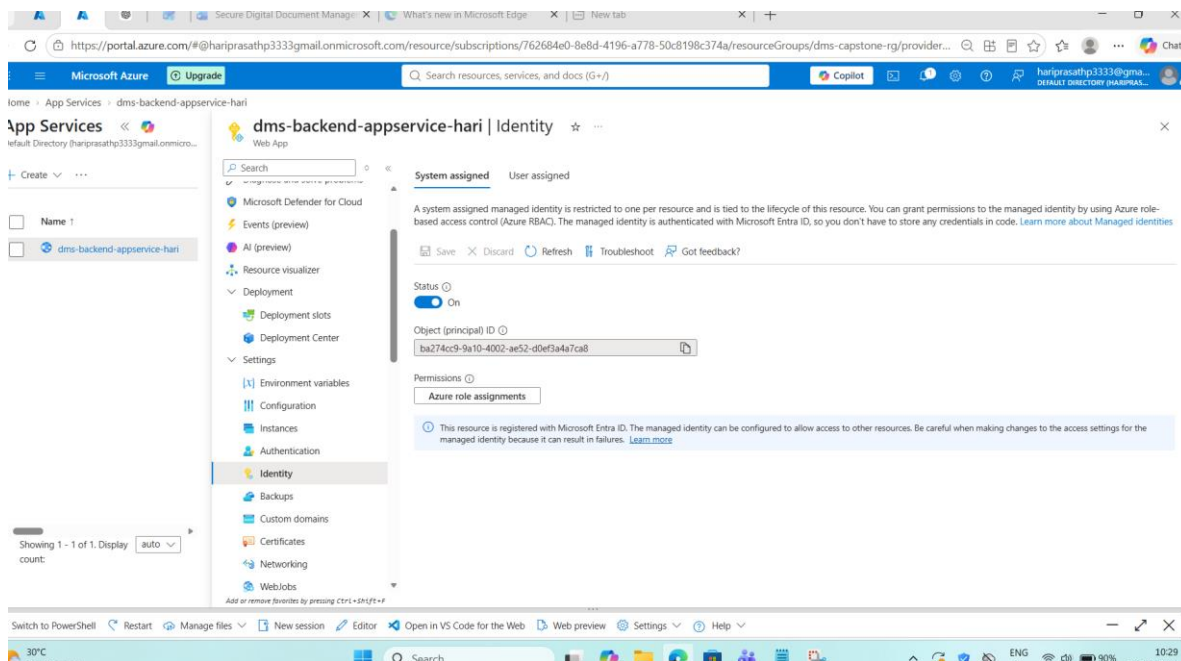


Figure 25. App Service system identity.

```
--resource-group dms-capstone-rg \
--account-name dms-cosmos-2026-hari \
--query "[].(Principal:principalId,RoleDefinition:roleDefinitionId,Scope:scope)" \
-o table
Principal                                RoleDefinition
Scope
-----
ba274cc9-9a10-4002-ae52-d0ef3a4a7ca8 /subscriptions/762684e0-8e8d-4196-a778-50c8198c374a/resourceGroups/dms-capstone-rg/providers/Microsoft.DocumentDB/databaseAccounts/dms-cosmos-2026-hari/sqlRoleDefinitions/0
00000000-0000-0000-0000-000000000000 /subscriptions/762684e0-8e8d-4196-a778-50c8198c374a/resourceGroups/dms-capstone-rg/providers/Microsoft.DocumentDB/databaseAccounts/dms-cosmos-2026-hari
hari [ ~ ]$
hari [ ~ ]$ az cosmosdb sql role assignment list \
--resource-group dms-capstone-rg \
--account-name dms-cosmos-2026-hari \
--query "[].(Principal:principalId,RoleDefinition:roleDefinitionId,Scope:scope)" \
-o table
Principal                                RoleDefinition
Scope
-----
ba274cc9-9a10-4002-ae52-d0ef3a4a7ca8 /subscriptions/762684e0-8e8d-4196-a778-50c8198c374a/resourceGroups/dms-capstone-rg/providers/Microsoft.DocumentDB/databaseAccounts/dms-cosmos-2026-hari/sqlRoleDefinitions/0
00000000-0000-0000-0000-000000000000 /subscriptions/762684e0-8e8d-4196-a778-50c8198c374a/resourceGroups/dms-capstone-rg/providers/Microsoft.DocumentDB/databaseAccounts/dms-cosmos-2026-hari
hari [ ~ ]$
```

Figure 26. Cosmos DB RBAC.

4.11 Key Vault

Azure Key Vault is used in the gateway certificate configuration. Application Gateway uses its managed identity to access the certificate secret, keeping certificate material out of the application source code.

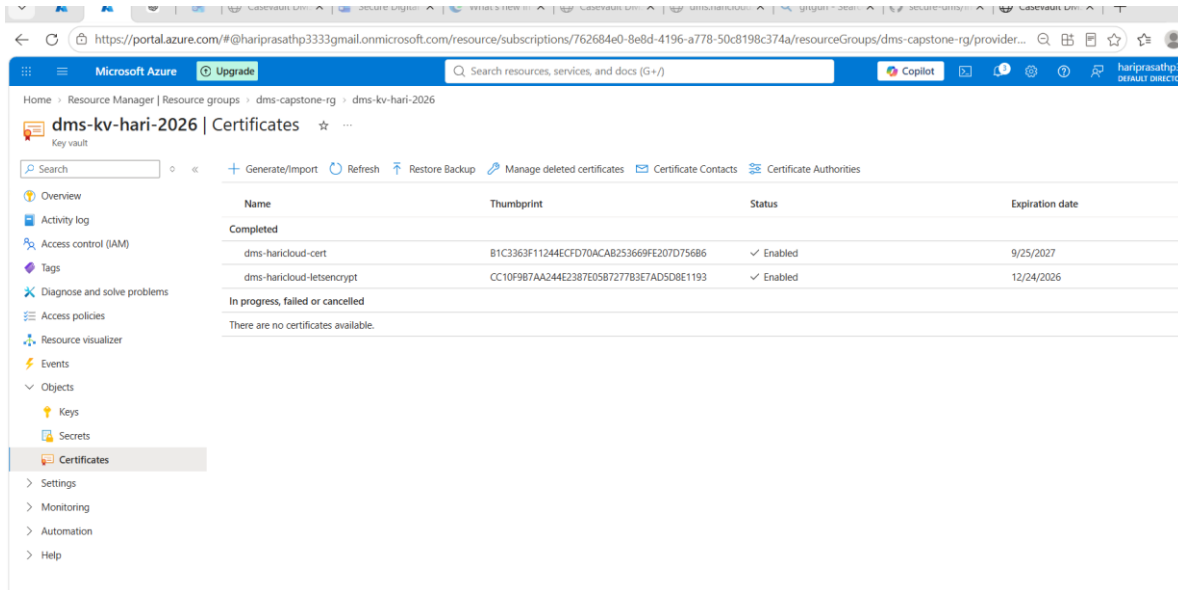


Figure 27. Key Vault certificate.

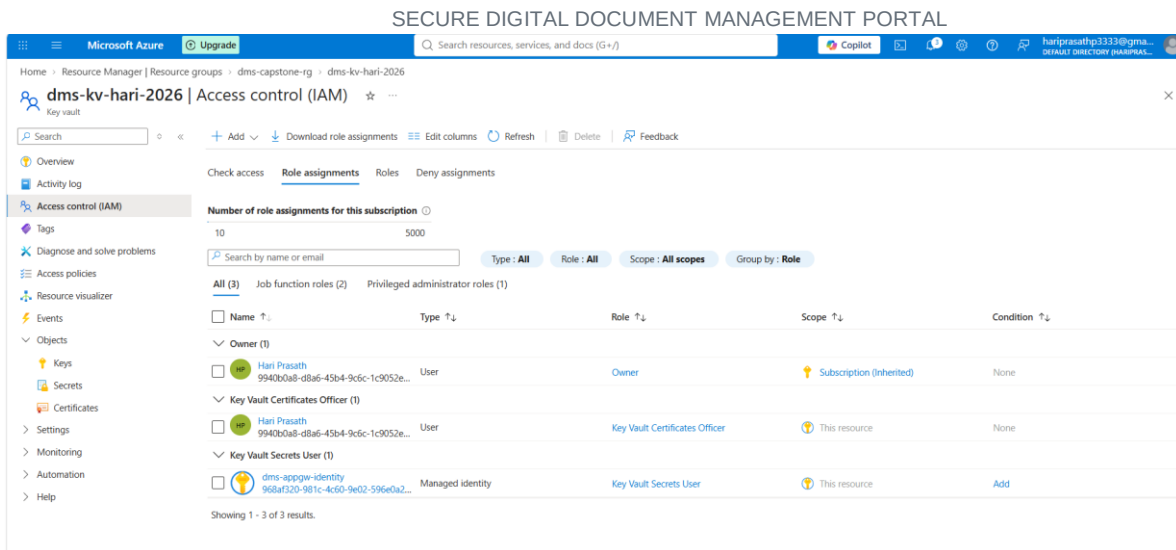


Figure 28. Key Vault RBAC.

4.12 Application Insights + Log Analytics

Application Insights provides application telemetry, while Log Analytics provides a central place for diagnostic logs and KQL queries. Diagnostic settings were configured for the relevant Azure resources so that operational information can be investigated.

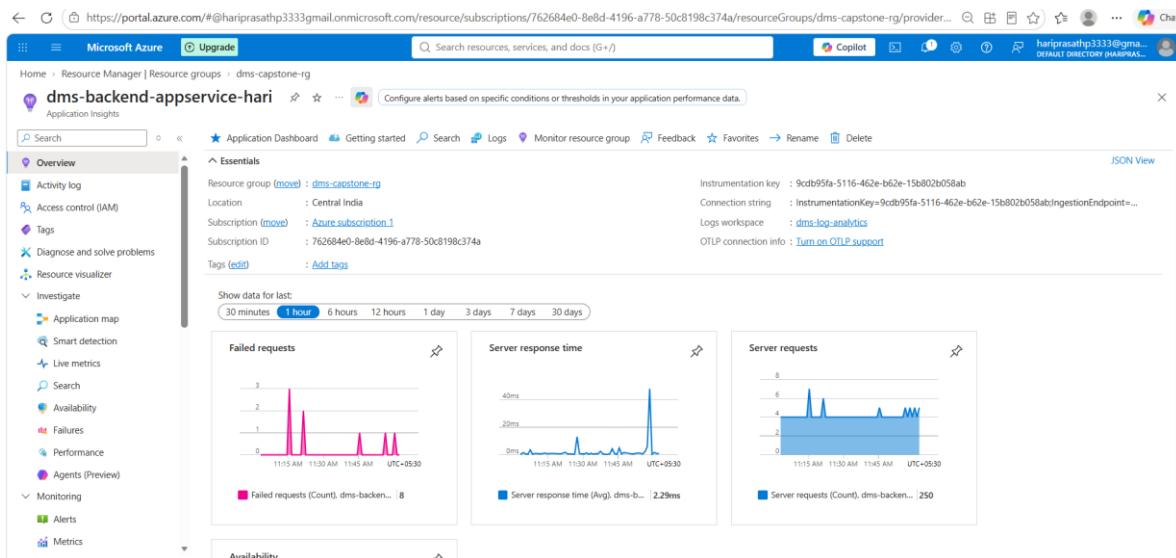


Figure 29. Application Insights.

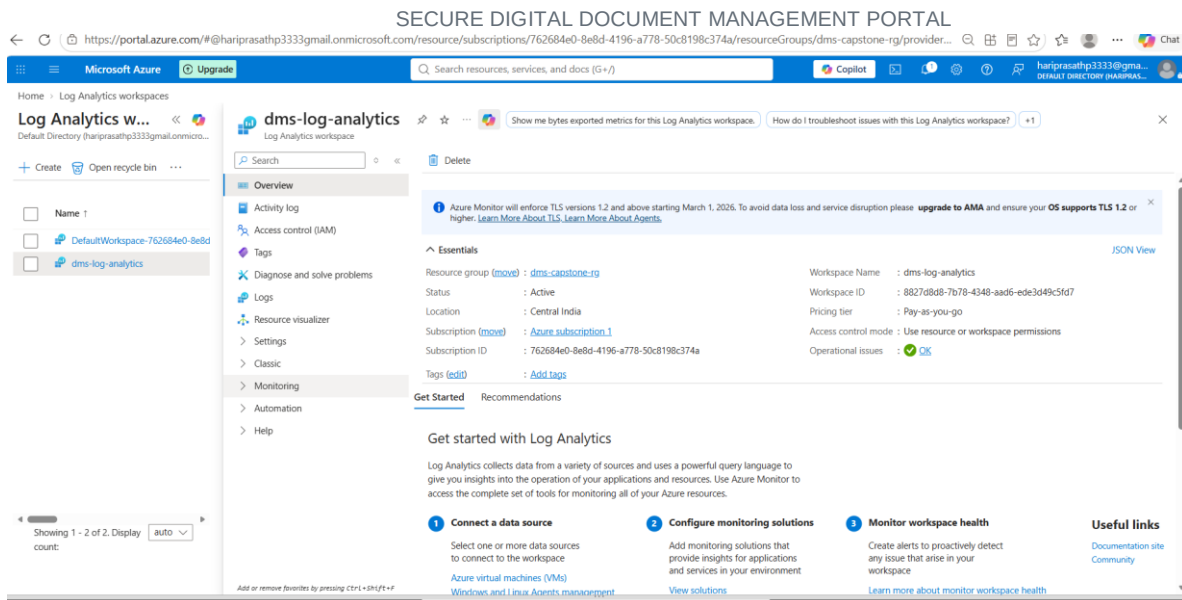


Figure 30. Log Analytics workspace.

4.13 Azure Monitor + Action Group

Azure Monitor is used for operational alerts. We configured an App Service 5xx alert and a Cosmos DB request-unit alert. The dms-alerts Action Group is configured to send email notifications.

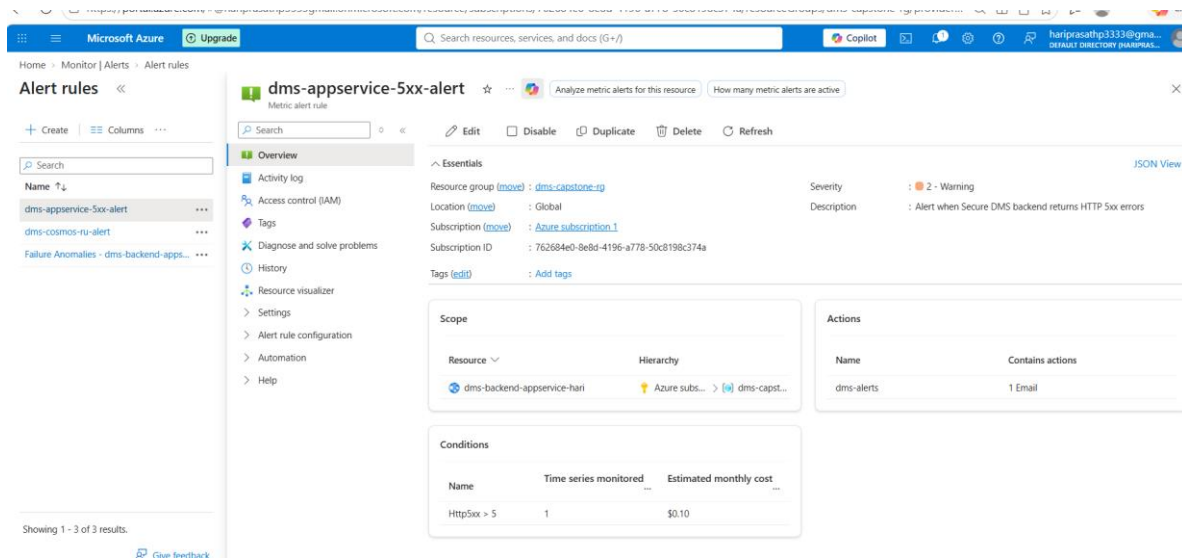


Figure 31. App Service 5xx alert.

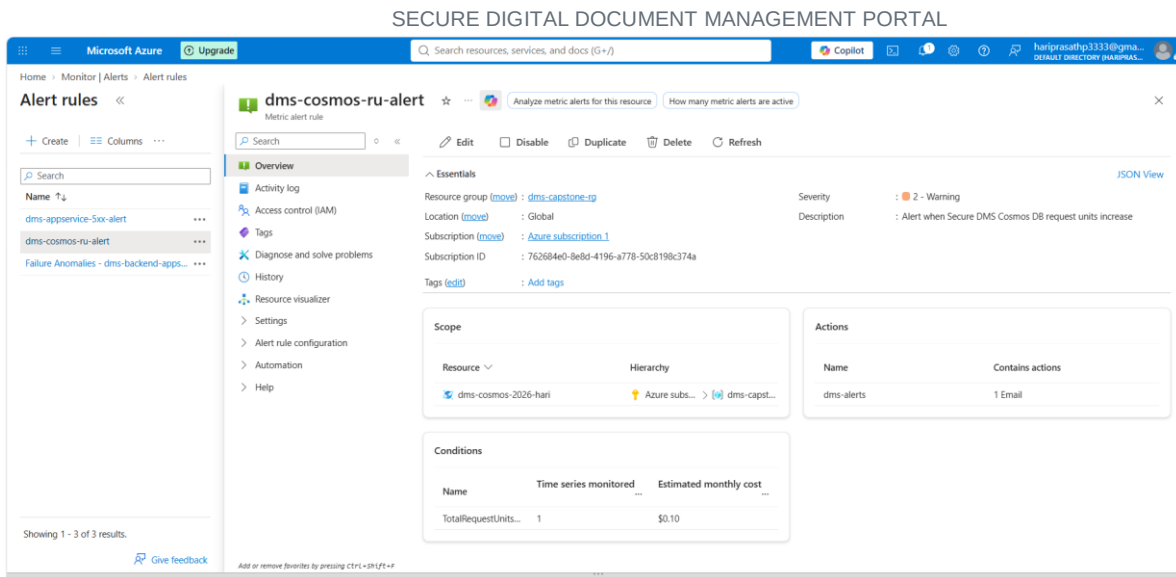


Figure 32. Cosmos DB RU alert.

4.14 GitHub Actions

GitHub Actions is the actual CI/CD platform used by the project. The workflows validate the backend, build the frontend, validate Bicep, deploy the backend to App Service and deploy the frontend to Azure Storage.

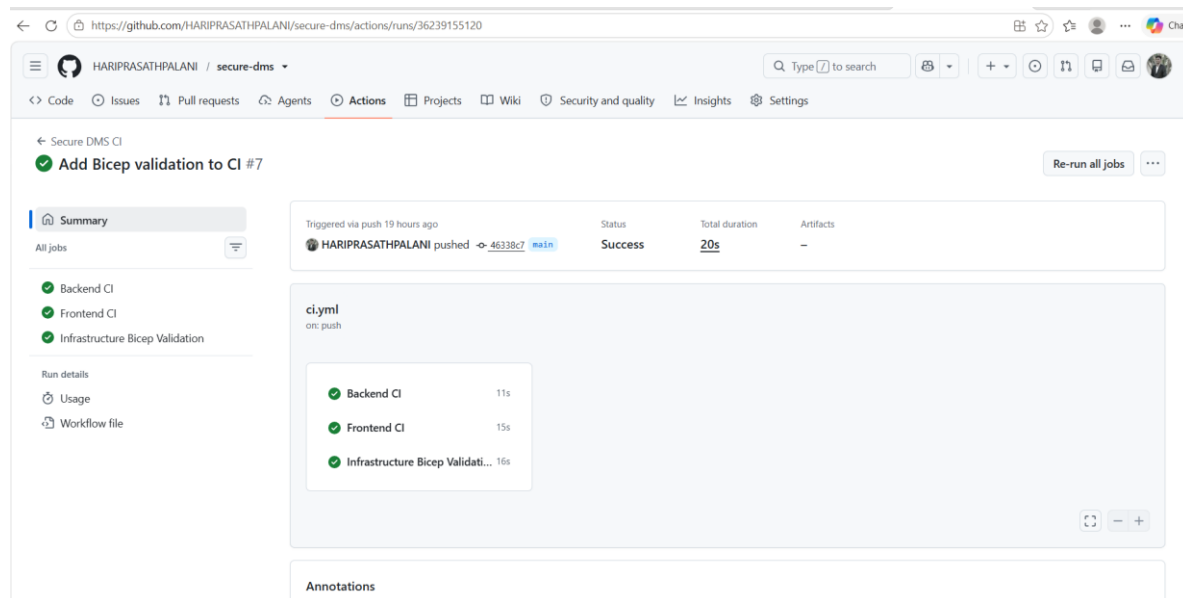


Figure 33. GitHub Actions workflow.

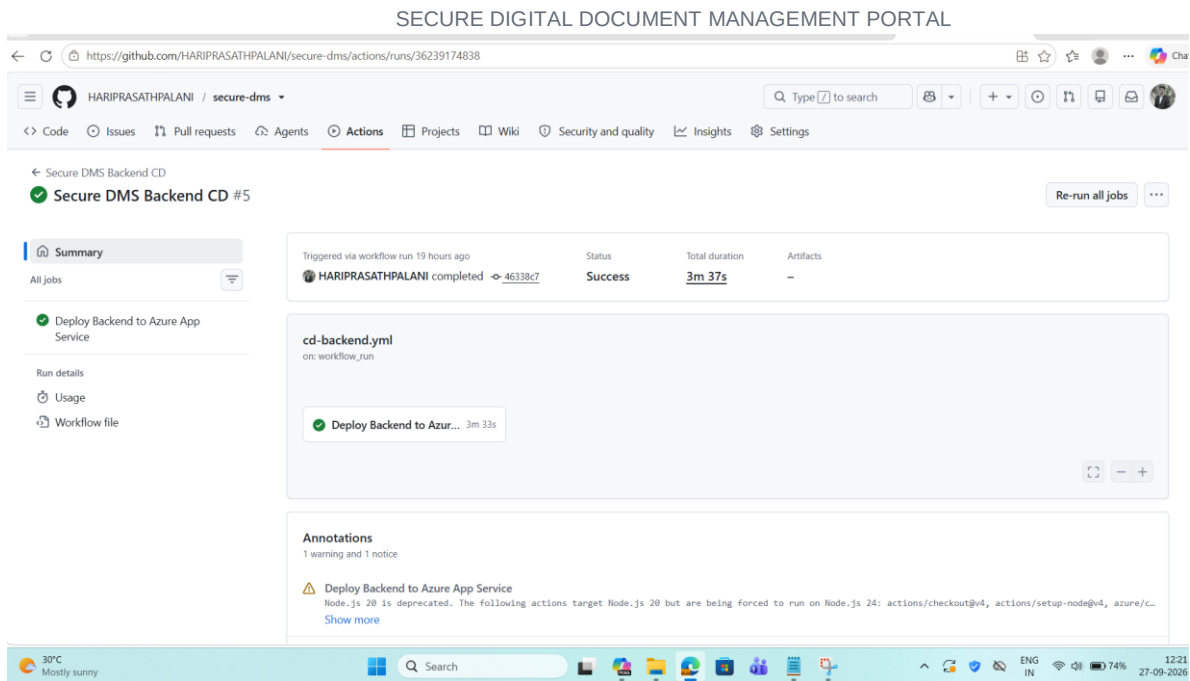


Figure 34. Backend deployment run.

4.15 Bicep

Bicep is used to represent the Azure infrastructure in code. The project is split into infrastructure modules and the CI workflow validates the Bicep file. The current modules safely reference the already deployed environment rather than claiming to recreate production from zero.

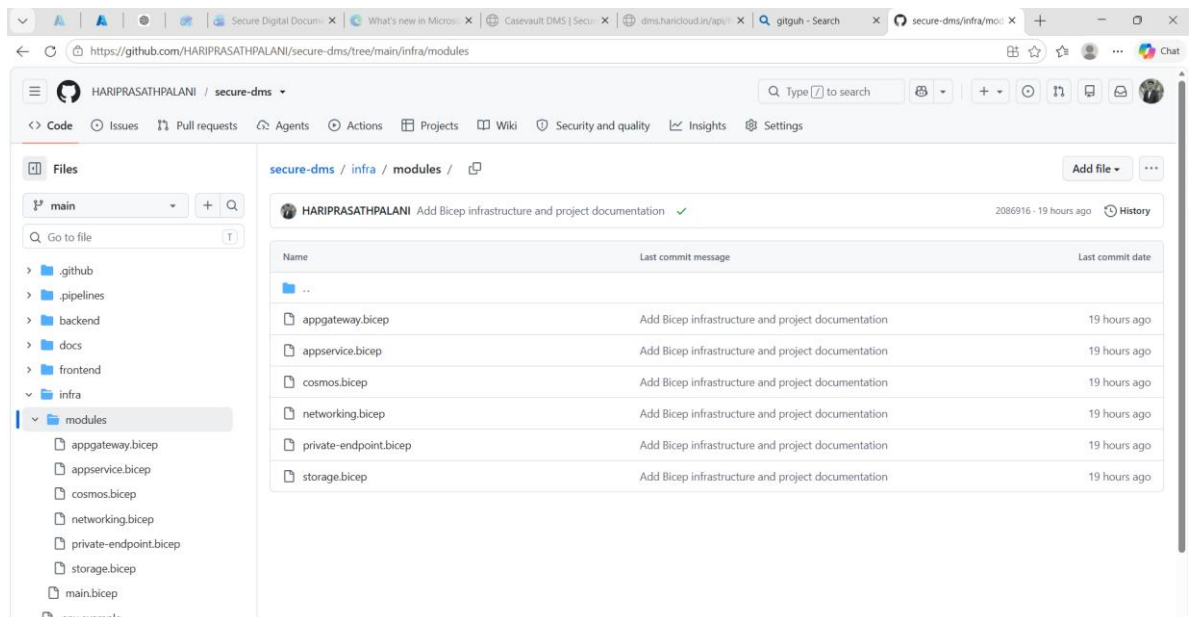


Figure 35. Bicep infrastructure structure.

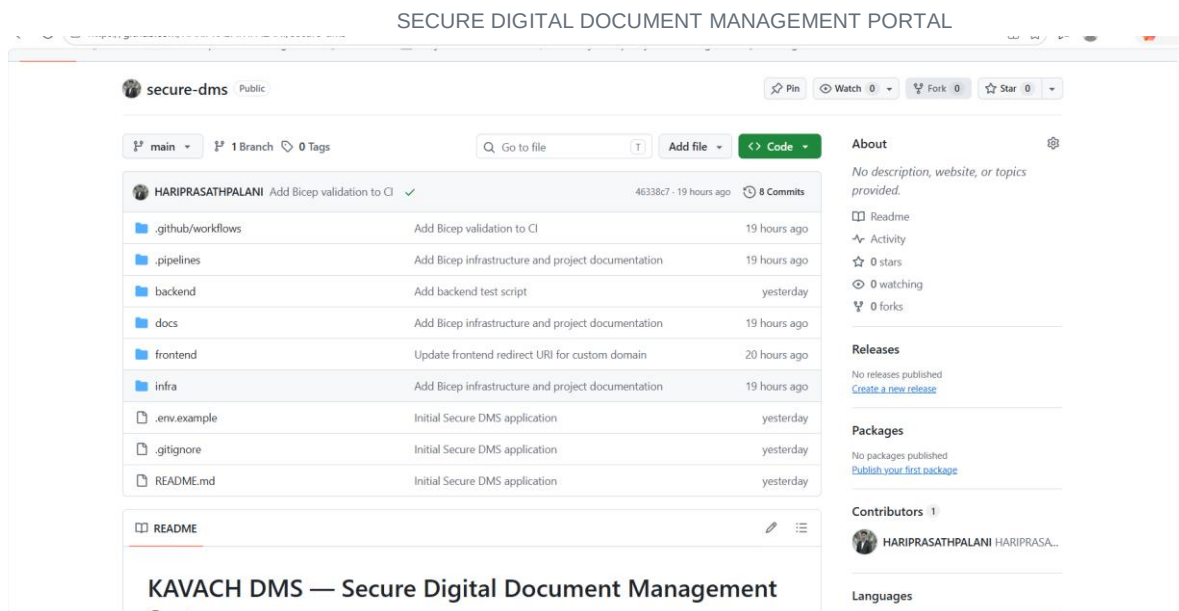


Figure 36. Bicep validation.

5. Document Management Flow

The most important business flow in Secure DMS is document management. The application deliberately separates the document file from the document information. The file goes to Blob Storage; the metadata goes to Cosmos DB.

1. The authenticated user selects a case and chooses a document file.
2. The backend verifies the request and checks that the case exists.
3. The file is uploaded to the Azure Blob Storage documents container.
4. The backend creates the document metadata record in the Cosmos DB Documents container.
5. An audit event is written for the document operation.
6. When the user downloads a document, the backend retrieves the metadata, checks the blob and streams the file back to the user.

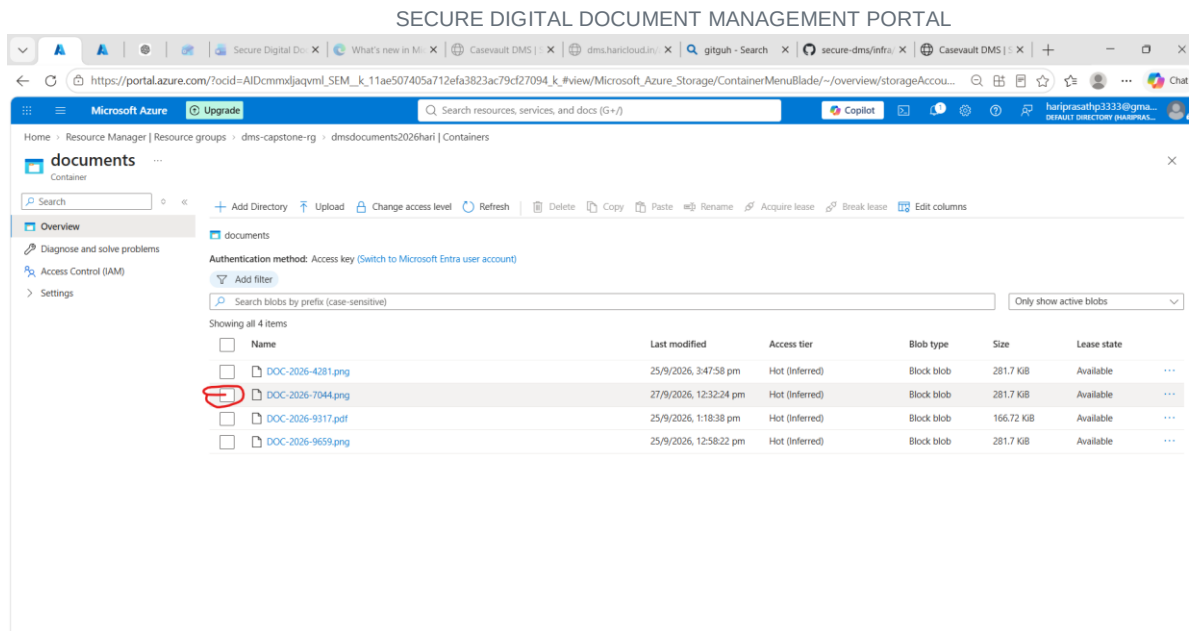


Figure 37. Document upload evidence.

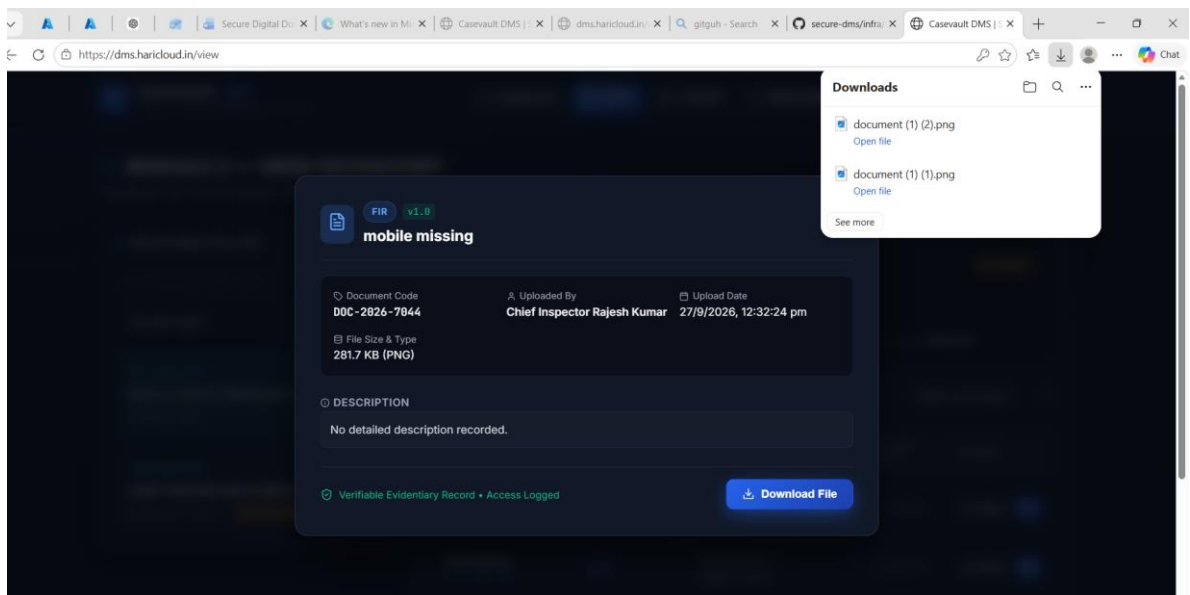


Figure 38. Document download evidence.

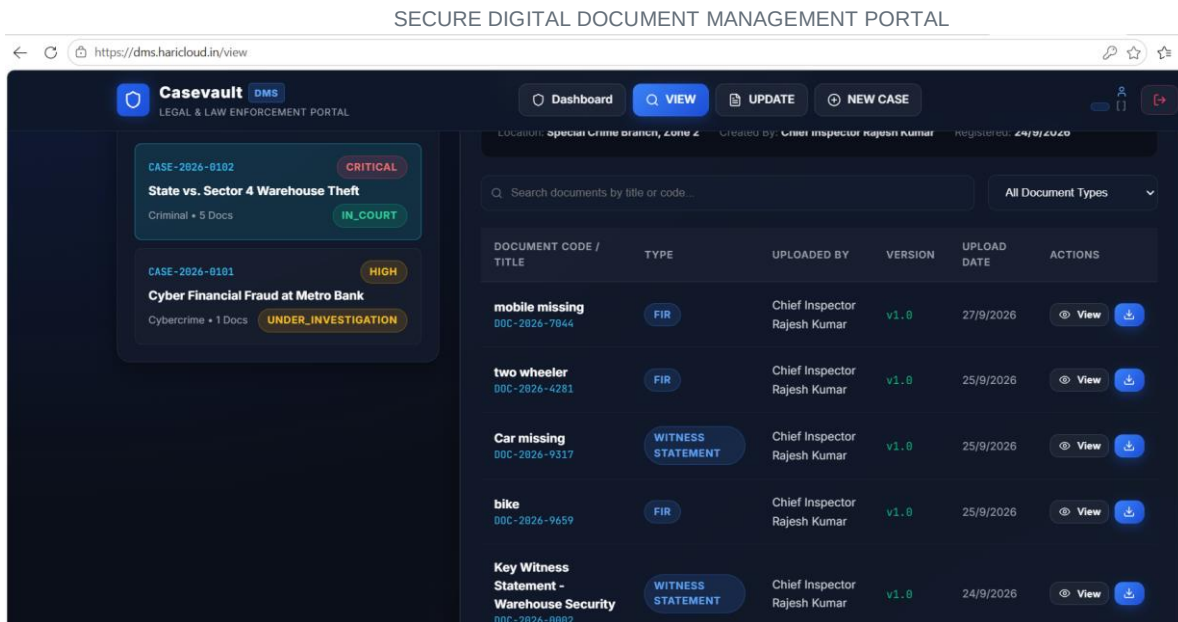


Figure 39. Audit trail evidence.

6. Security Design

Security in Secure DMS is layered. Authentication starts with Microsoft Entra ID, the public edge is protected by HTTPS and WAF, the backend has access restrictions and VNet integration, Cosmos DB is private, and Azure resource access uses Managed Identity and RBAC.

- Microsoft Entra ID protects user sign-in.
- MSAL obtains the access token used for backend calls.
- Backend JWT validation prevents unauthenticated API access.
- Application Gateway provides HTTPS termination and WAF protection.
- App Service access restrictions limit direct backend exposure.
- Cosmos DB public network access is disabled.
- Cosmos DB uses a private endpoint and private DNS.
- Managed Identity removes the need for application-stored Azure service keys.
- Sensitive UPDATE and NEW CASE operations use step-up re-authentication.
- Audit logs record important security and document events.

SECURE DIGITAL DOCUMENT MANAGEMENT PORTAL

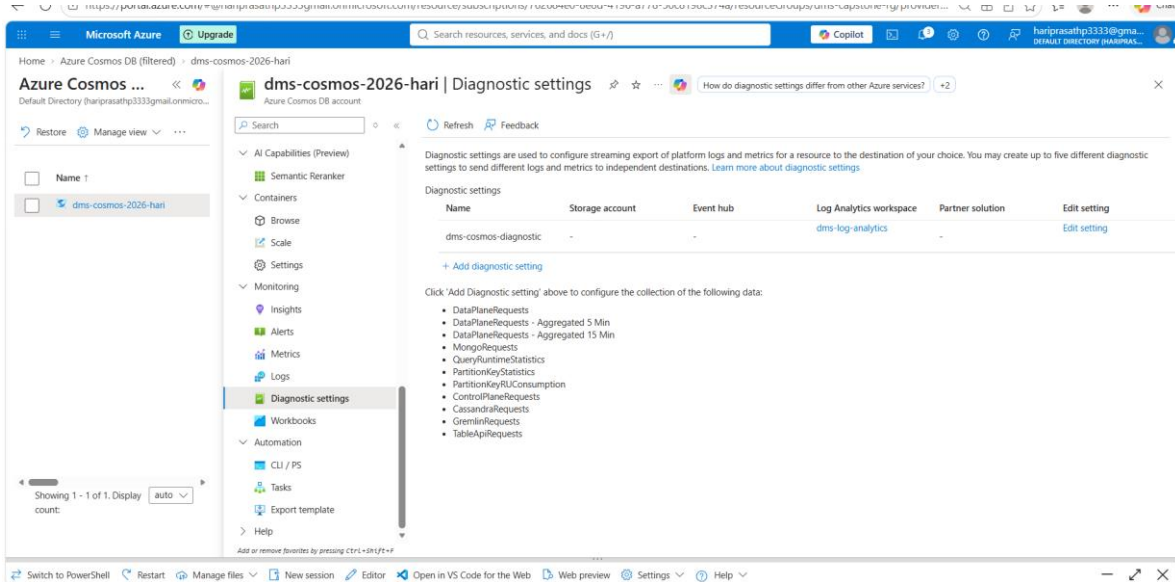


Figure 40. Cosmos DB diagnostic configuration.

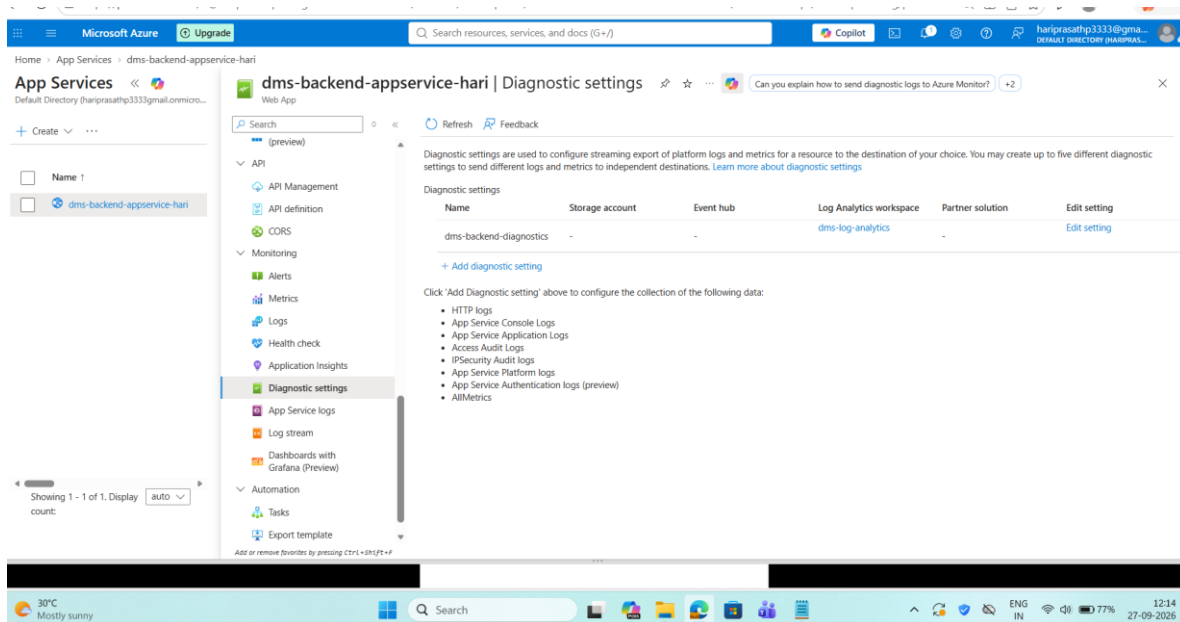


Figure 41. App Service diagnostic settings.

7. Monitoring and Operational Visibility

Monitoring was added so that the application can be observed after deployment instead of being treated as a one-time demo. Application Insights captures application telemetry, Log Analytics stores diagnostic information, and Azure Monitor evaluates important metrics.

- Application Insights monitors application requests and failures.
- App Service diagnostics send logs to Log Analytics.
- Application Gateway diagnostics capture access, performance and WAF-related information.
- Cosmos DB diagnostics are sent to Log Analytics.

- A KQL query is used to inspect recent App Service HTTP activity.
- An App Service 5xx alert and a Cosmos DB RU alert provide operational thresholds.
- The Action Group is configured for email notification.

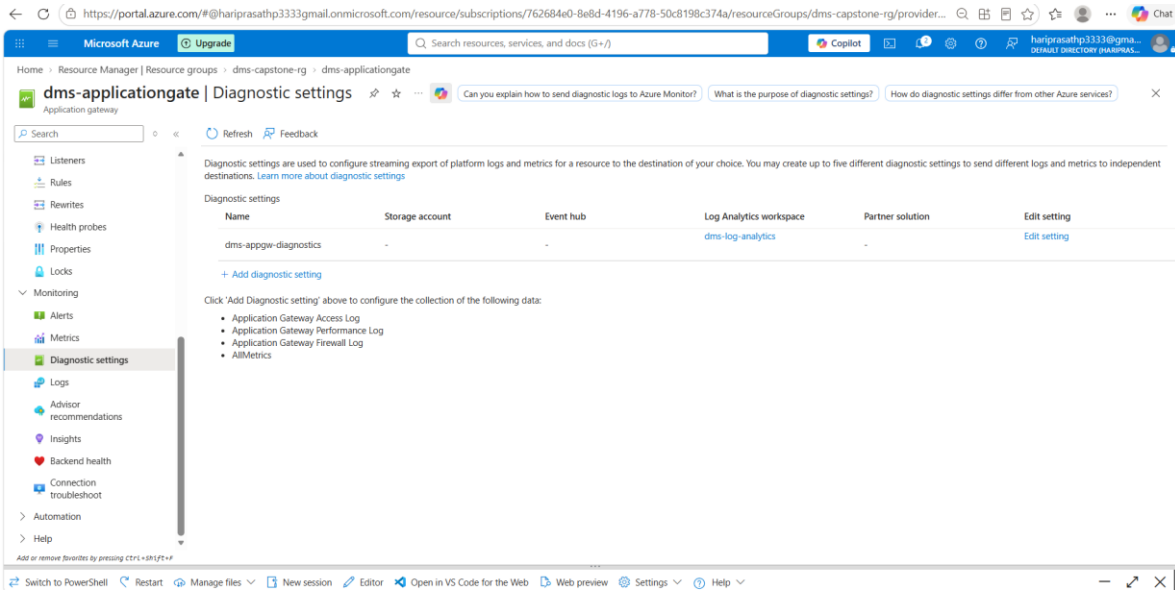


Figure 42. Application Gateway diagnostic settings.

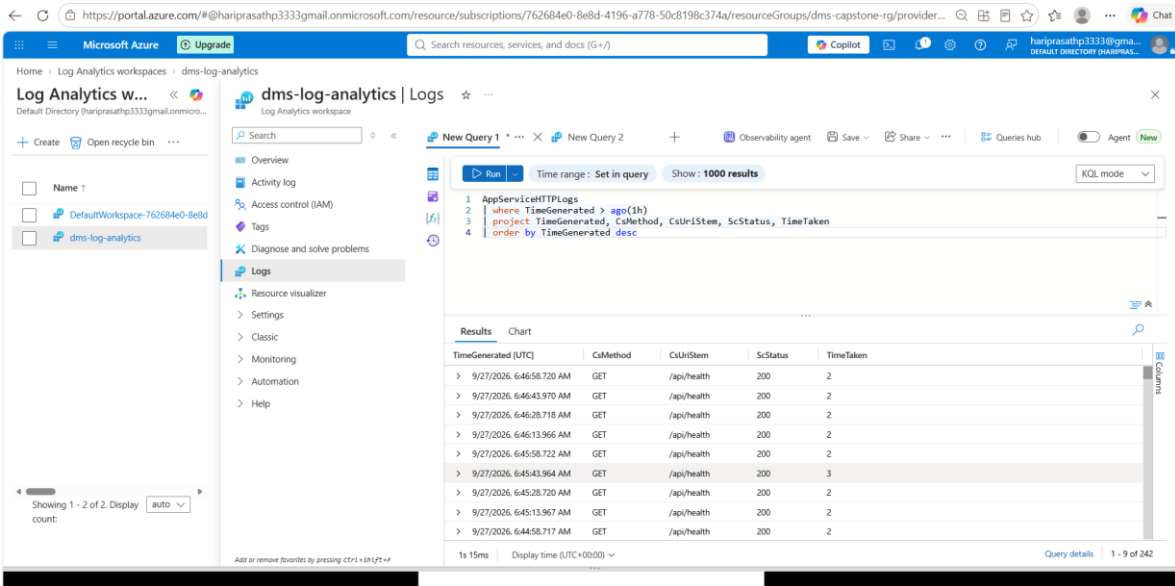


Figure 43. KQL traffic query.

8. Deployment and CI/CD

The project source is maintained in GitHub. GitHub Actions performs the automated validation and deployment work. The workflow keeps application delivery repeatable and reduces manual deployment steps.

8.1 CI

- Backend dependencies are installed and backend tests are executed.
- Frontend dependencies are installed and the React production build is generated.
- Bicep is built/validated in the workflow.

8.2 CD

- The backend deployment targets Azure App Service.
- The frontend deployment uploads the built dist output to the Azure Storage \$web container.
- Azure authentication is performed using the project's GitHub Actions identity configuration.

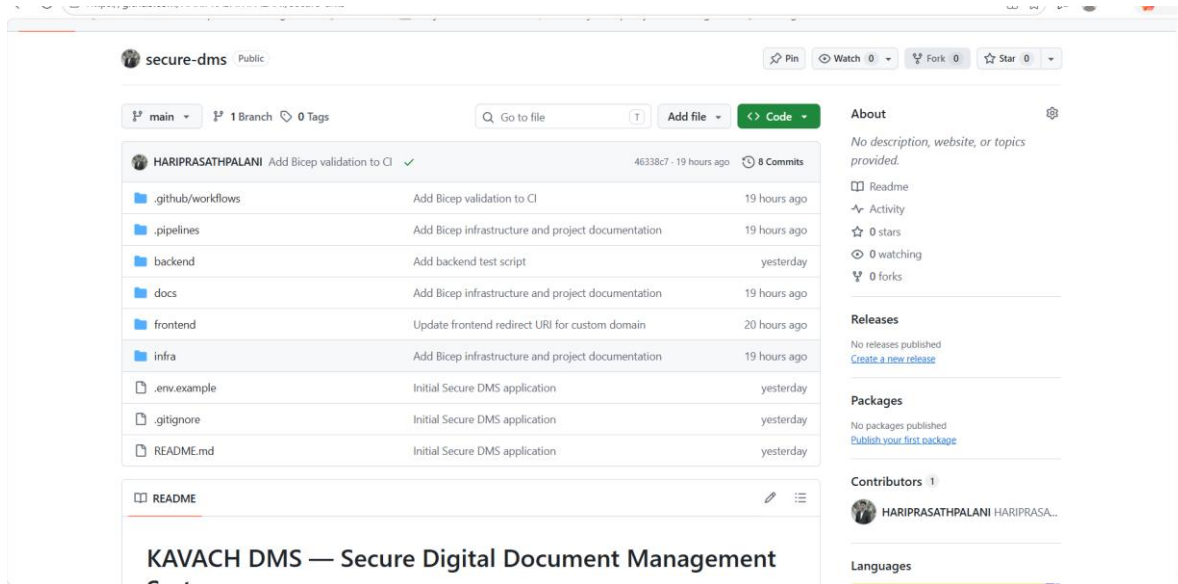
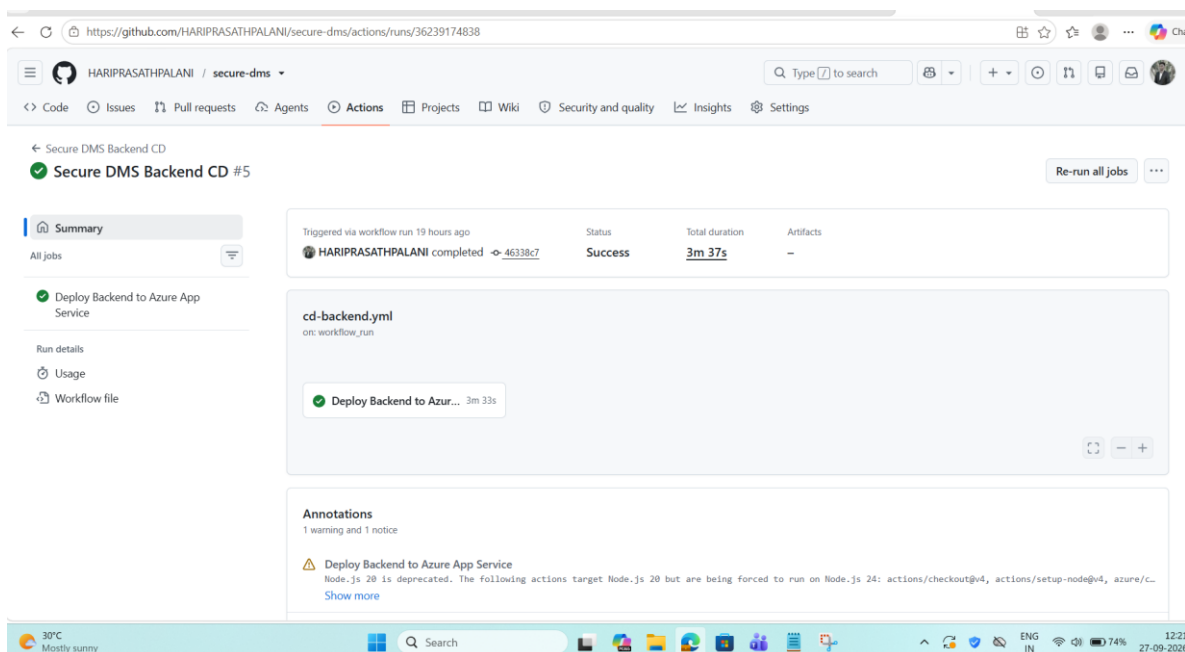


Figure 44. Secure DMS GitHub repository structure.



SECURE DIGITAL DOCUMENT MANAGEMENT PORTAL
Figure 45. Successful backend CD.

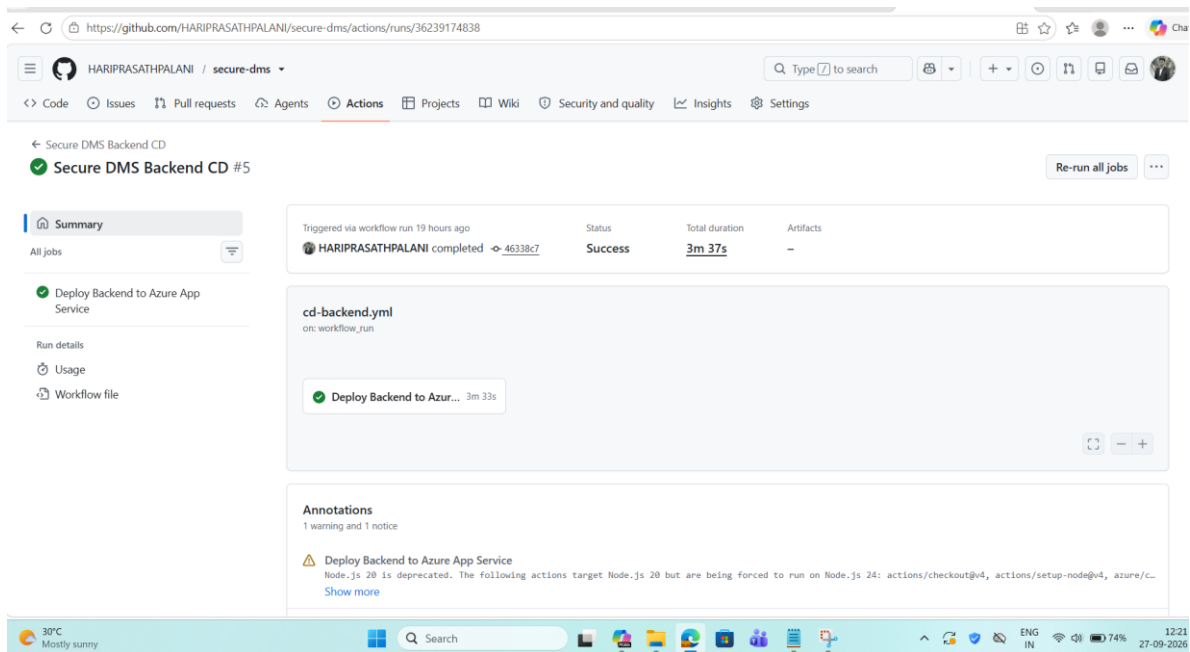


Figure 46. Successful frontend CD.

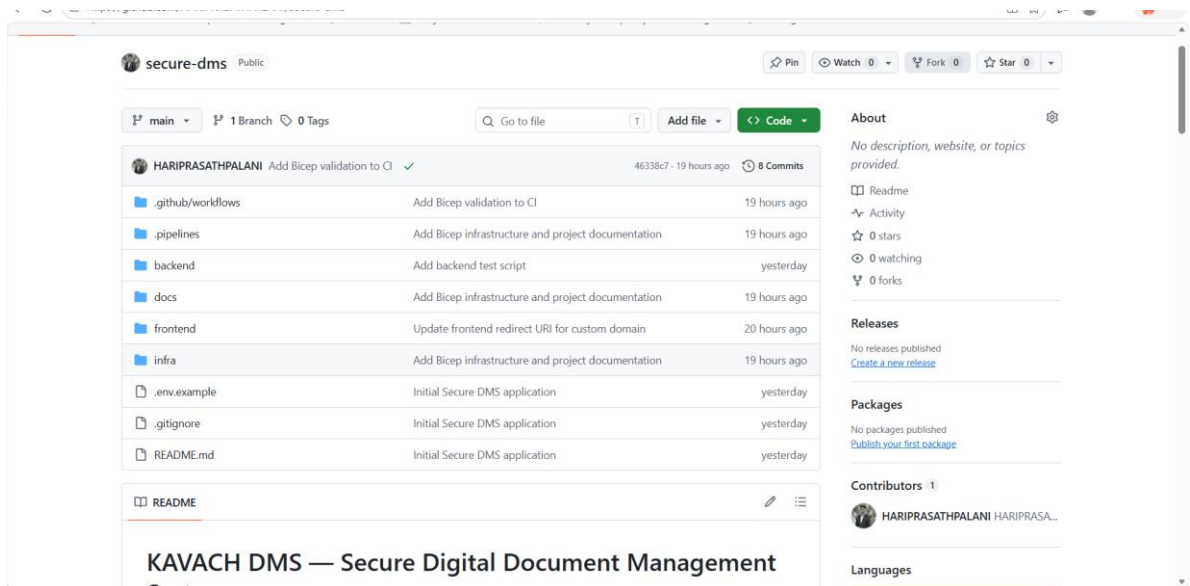


Figure 47. Successful Bicep validation.

9. Problems Faced During Deployment — and How We Fixed Them

The deployment was not a straight one-click process. Several issues appeared while moving the project into Azure. The useful part of the experience was identifying whether an error belonged to the application, the local environment, Azure networking, or an Azure command.

SQLite / node-gyp rebuild error

What happened: The earlier local Secure DMS setup used SQLite and the native sqlite3 package. The environment did not have a suitable native build toolchain, so node-gyp rebuild failed.

How it was rectified: The deployment direction was moved to the Azure-hosted implementation and the production architecture uses Cosmos DB for application data, avoiding dependence on the local SQLite native build path.

Missing node_modules in Cloud Shell

What happened: When the backend was opened in a fresh Cloud Shell environment, packages such as express and @azure/cosmos were not available because node_modules had not been installed.

How it was rectified: The project dependencies were treated as environment dependencies and the deployment workflow uses npm install/npm ci before running or building the application.

Cosmos DB CLI query command unavailable

What happened: An attempted az cosmosdb sql container query command was rejected because that subcommand is not available in the Azure CLI command set.

How it was rectified: Instead of depending on a non-existent CLI subcommand, the database was handled through the application SDK, Cosmos Data Explorer where appropriate, and the deployed API.

Cosmos RBAC assignment issue

What happened: During Managed Identity setup, the role assignment command required the correct Cosmos native SQL role definition and scope.

How it was rectified: The backend Managed Identity was assigned Cosmos DB Built-in Data Contributor at the Cosmos account scope, and the duplicate assignment created during troubleshooting was cleaned up.

Application Gateway diagnostics initially empty

What happened: The Application Gateway Diagnostic Settings blade initially showed that no diagnostic setting had been defined.

How it was rectified: A dedicated diagnostic setting was created to send gateway logs and metrics to the dms-log-analytics workspace.

Cosmos diagnostic settings needed configuration

What happened: The Cosmos DB Diagnostic Settings page required the log categories, destination workspace and resource-specific destination to be selected.

How it was rectified: The Cosmos diagnostic setting was configured to send the required logs to dms-log-analytics.

Action Group synthetic test limitation

What happened: A synthetic Action Group test returned a conflict indicating that the subscription did not support the requested test operation.

How it was rectified: The Action Group itself remained configured and enabled, and the alert rules were linked to it. The limitation was treated as a platform test restriction rather than an application failure.

Backend direct-access protection

What happened: The backend had to be protected so that the normal public path goes through Application Gateway rather than exposing App Service directly.

How it was rectified: App Service access restrictions were configured to allow the Application Gateway subnet while denying unmatched inbound traffic.

10. Final Azure Implementation

After the individual components were configured, the services were connected into one working solution. The final environment consists of an authenticated React frontend, an App Service backend, private Cosmos DB connectivity, Blob Storage document storage, Application Gateway with WAF and HTTPS, Azure identity and RBAC, monitoring, and automated delivery.

- Frontend — deployed as a static website in Azure Storage.
- Identity — Microsoft Entra ID with frontend and backend application registrations.
- Gateway — Application Gateway with HTTPS, routing, health probe and WAF.
- Backend — Node.js/Express API on Azure App Service with VNet Integration and access restrictions.
- Database — Cosmos DB with public access disabled, private endpoint and Managed Identity RBAC.
- Documents — Azure Blob Storage for actual files and Cosmos DB for metadata.
- Security — Managed Identity, RBAC, private networking and step-up re-authentication.
- Monitoring — Application Insights, Log Analytics, Azure Monitor alerts and Action Group.
- Delivery — GitHub Actions CI/CD and Bicep validation.

11. Project Outcome

The completed Secure DMS is more than a website hosted on Azure. It is an end-to-end cloud application in which the user interface, identity, API, data, document storage, network controls, gateway security, monitoring and deployment process work together. The implementation also provides a clear explanation for why each Azure service exists and what problem it solves.

For the capstone evaluation, the architecture can be explained from the outside in: user → Entra ID → Application Gateway → frontend/backend → private data services. That flow is the central story of the project and connects the application features to the Azure services underneath them.

12. Alignment with the Azure Capstone Brief

The supplied capstone brief asks for a static frontend in Azure Storage, Entra-based authentication, a VNet-integrated backend without unrestricted public access, private Cosmos DB connectivity, a Layer-7 entry point, monitoring, and DevOps/IaC. Secure DMS implements these major architectural areas. The brief also permits GitHub Actions as a CI option, and the actual project uses GitHub Actions rather than replacing the real implementation with Azure DevOps terminology.

The report therefore documents the project as it was actually built, while using the capstone brief as the architectural reference. It does not present optional capstone services as if they were deployed.

13. Conclusion

Secure Digital Document Management Portal demonstrates how a practical web application can be converted into a secure Azure cloud solution. The React frontend provides the user experience, Node.js/Express provides the business API, Cosmos DB and Blob Storage manage application and document data, and Azure networking, identity, gateway security and monitoring provide the cloud foundation.

The main technical lesson from the project is that cloud architecture is not only about deploying individual services. The value comes from connecting those services correctly: identity controls access, Application Gateway controls entry, App Service performs the business logic, private networking protects the data path, Managed Identity controls Azure-to-Azure access, and monitoring provides visibility after deployment.

Documentation note: The screenshots are intentionally placed beside the relevant explanation instead of being collected into a separate evidence index.